OficioYa

DOCUMENTO DE ARQUITECTURA · HIGH LEVEL DESIGN

Arquitectura del backend

API REST de la plataforma OficioYa: el talento de tu zona a un click de distancia.

PROYECTO
OficioYa · Directorio inteligente y reputación digital para el trabajo informal

CURSO
Desarrollo y Operaciones de Software (DOSW) · Escuela Colombiana de Ingeniería Julio Garavito

VERSIÓN
1.0 · 20 de octubre de 2026 · Sprint 4

CÓDIGO DESCRITO
OficioYa-Backend, rama `develop`, commit `890b8b8` (19-oct-2026, PR #36)

INTEGRANTES
Carlos Sánchez @Carlossj8
Oscar Lasso @Oscar10lm
Cristian Santiago Moreno Ruiz @cristianmorenor
José Alejandro Martínez Arias @JoseMartinez883
Juan Diego Gaitán @mamagege



Historial de revisión

Versión	Fecha	Autor	Cambios
1.0	20-oct-2026	Equipo OficioYa	Primera versión completa del documento de arquitectura del backend para el Sprint 4: descripción, tecnologías, funcionalidades, diagramas actualizados al código, los 60 endpoints expuestos, manejo de errores, secuencias, despliegue, evidencias y limitaciones conocidas.

Contenido

1. Descripción del servicio
2. Tecnologías utilizadas
3. Funcionalidades y casos de uso
4. Diagramas de arquitectura
5. Funcionalidades expuestas (API)
6. Manejo de errores
7. Diagramas de secuencia
8. Despliegue y seguridad
9. Calidad y evidencias
10. Limitaciones conocidas
11. Bibliografía

Cómo leer este documento

Este documento describe el backend **tal como está en el código**, no como quisiéramos que estuviera. Cada diagrama se dibujó de nuevo a partir de las clases, entidades y controladores reales, y cada afirmación de calidad viene de una ejecución hecha el 20 de octubre de 2026. Lo que todavía falta está declarado en la sección 10, con su impacto y cómo lo resolveríamos.



1. Descripción del servicio

1.1 Qué es y qué problema resuelve

Cuando a alguien se le revienta un tubo a las diez de la noche casi nunca sabe a quién llamar. Termina preguntando en el grupo de WhatsApp del conjunto o marcando el primer número pegado en un poste, sin saber si esa persona es de confianza, si cobra justo o si va a llegar. Del otro lado está el plomero, la electricista o el cerrajero que trabaja por su cuenta: es bueno en lo suyo, pero vive del voz a voz y no tiene cómo demostrar su experiencia fuera de su barrio.

OficioYa conecta a esas dos personas. El backend es la API REST que sostiene todo el producto: guarda las cuentas y los perfiles profesionales, ofrece el directorio con su buscador, gobierna el ciclo de vida de cada servicio, calcula la reputación a partir de reseñas que solo puede dejar quien de verdad recibió o prestó el servicio, y le da al administrador las herramientas para cuidar la comunidad.

1.2 Para quién

Actor	Quién es	Qué hace con la API
Contratante	Hogares, estudiantes, adultos mayores y comercios que necesitan resolver un daño o un mantenimiento con rapidez.	Busca trabajadores, crea y envía solicitudes, chatea, califica y reporta.
Trabajador independiente	Técnicos y personas de oficios prácticos de 18 a 65 años que quieren hacerse visibles en su zona.	Arma su perfil, activa "Disponible ahora", responde solicitudes, sube documentos de verificación y califica a sus clientes.
Administrador	Persona del equipo de OficioYa.	Revisa reportes y pausa cuentas.
Cliente web (SPA)	La aplicación React del repositorio OficioYa-Frontend.	Consume la API por HTTPS enviando el JWT en cada petición.

Una misma cuenta puede usar los dos modos, Contratante y Profesional, y pasar de uno a otro con `/auth/switch-role`.

1.3 Principios que guían el diseño

- **Mérito, no pago.** El orden de los resultados depende solo de la reputación o de la cercanía. No existe ningún campo de pago publicitario en el modelo.
- **Confianza verificable.** Solo se califica una solicitud COMPLETADA, las reseñas reportadas salen del promedio y un trabajador con tres o más reseñas y promedio menor a 3,0 pierde el modo "Disponible ahora".
- **Monolito modular en capas.** Controlador → Servicio → Validador → Repositorio, con interfaces en cada frontera para poder cambiar implementaciones sin tocar el negocio.
- **Persistencia híbrida.** PostgreSQL para lo transaccional y MongoDB para lo flexible (portafolios y mensajes).
- **Seguridad sin estado.** JWT firmado en cada petición y autorización declarativa por método.

1.4 Diagrama de contexto

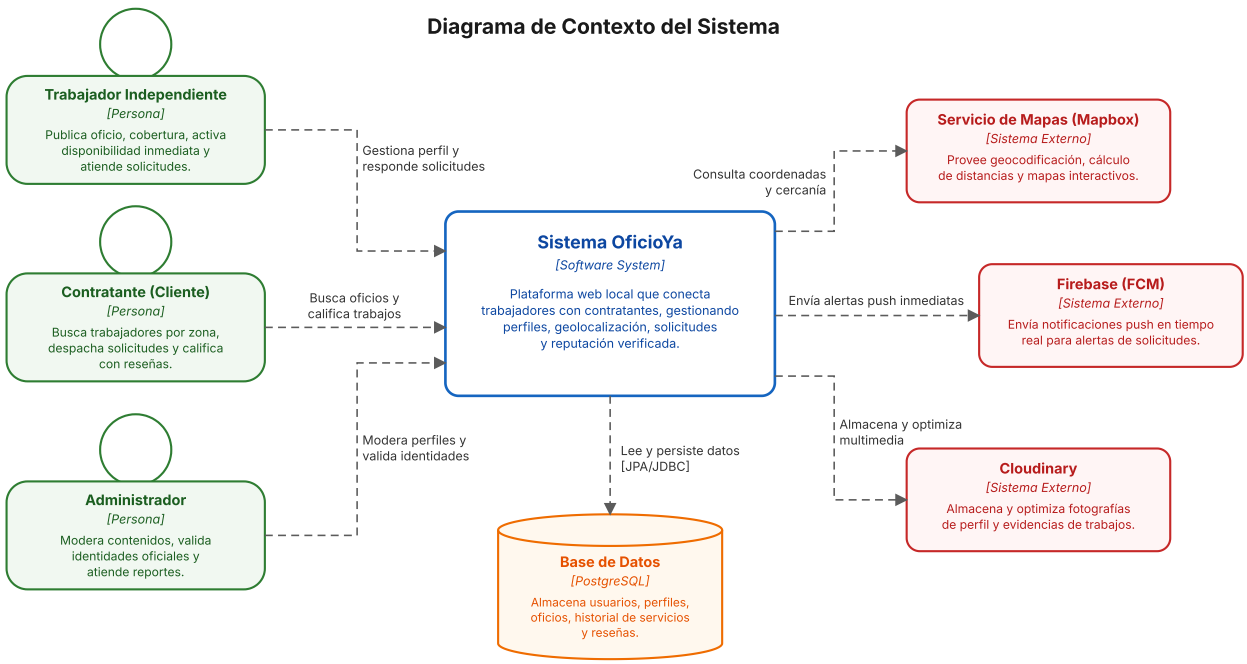


Figura 1. Diagrama de contexto: el backend como una caja, con sus actores y sistemas externos.

Es el mismo diagrama de contexto del README del repositorio. El sistema aparece como una sola caja. En verde están las tres personas que lo usan: el trabajador independiente, el contratante y el administrador, cada una con lo que hace en la plataforma. En rojo están los tres sistemas externos (Mapbox, Firebase Cloud Messaging y Cloudinary) y en naranja la base de datos PostgreSQL, a la que el sistema llega por JPA/JDBC. MongoDB, que guarda portafolios y mensajes, se detalla en los diagramas de componentes y de despliegue. En el código, los tres servicios externos están detrás de una interfaz (`IMapAdapter`, `INotificacionStrategy`, `IStorageService`) y hoy usan implementaciones simuladas (L9); cuando se conecten los proveedores reales, el resto del sistema no cambia.

2. Tecnologías utilizadas

Las versiones salen del `pom.xml`, del `Dockerfile`, del `docker-compose.yml` y del pipeline de GitHub Actions.

Capa	Tecnología	Versión	Para qué la usamos
Lenguaje	Java	21 LTS	Lenguaje de todo el backend. Records para DTOs inmutables y API moderna de fechas.
Framework	Spring Boot	3.4.1	Arranque, inyección de dependencias, servidor Tomcat embebido y autoconfiguración.
Web	Spring Web MVC	6.2	Controladores REST, binding y negociación JSON.
Validación	Jakarta Bean Validation (Hibernate Validator)	3.x	Reglas declarativas en los DTOs: `@NotBlank`, `@Size`, `@Email`, `@DecimalMin`…
Seguridad	Spring Security	6.4.2	Cadena de filtros sin estado, BCrypt y autorización por método con `@PreAuthorize`.
Tokens	JJWT (io.jsonwebtoken)	0.11.5	Firma y validación de JWT con HMAC-SHA256.
Persistencia SQL	Spring Data JPA + Hibernate	3.4 / 6.6	Repositorios, entidades y consultas dinámicas con `Specification` (Criteria API).
Base relacional	PostgreSQL	15	Datos transaccionales: usuarios, perfiles, oficios, solicitudes, reseñas, reportes y referidos.
Persistencia NoSQL	Spring Data MongoDB	4.4	Repositorios de documentos para portafolios y mensajes.
Base documental	MongoDB	6	Colecciones `portafolios` y `mensajes`.
Base de pruebas	H2	2.x	Base en memoria para la suite de pruebas.
Mapeo	MapStruct	1.5.5	Conversión DTO ⇄ dominio ⇄ entidad generada en compilación.
Productividad	Lombok	1.18	Getters, setters, builders y constructores.
Documentación	springdoc-openapi (Swagger UI)	2.8.4	Contrato OpenAPI 3 en `/v3/api-docs` y consola en `/swagger-ui/index.html` con esquema bearerAuth.
Pruebas	JUnit 5 · Mockito · Spring Security Test	—	Pruebas unitarias con patrón AAA y pruebas de autorización con MockMvc.
Cobertura	JaCoCo	0.8.11	Reporte de cobertura y regla mínima del 85 % de instrucciones en el `pom.xml`.
Construcción	Maven Wrapper	3.9	Compilación reproducible con `./mvnw`.
Contenedores	Docker · Docker Compose	24+ · 3.8	Imagen multietapa (Temurin 21) y entorno local con PostgreSQL y MongoDB.
CI/CD	GitHub Actions	—	Pipeline de build, pruebas, análisis y despliegue en cada push a `develop` o `main`.
Nube	Azure App Service (Linux)	—	Hospeda el JAR en `oficioya-backend.azurewebsites.net`.

> **¿Por qué dos bases de datos?**
> Una solicitud de servicio es un acuerdo entre dos personas con efectos en su reputación, así que necesita transacciones ACID e integridad referencial: eso lo da PostgreSQL. Los portafolios y los mensajes cambian de forma según el oficio y se leen como bloques completos, así que encajan mejor como documentos en MongoDB. Ambas se enlazan por identificadores (patrón referenciado), sin claves foráneas entre motores.

3. Funcionalidades y casos de uso

La tabla resume las funcionalidades implementadas, agrupadas por módulo. Cada fila lleva el identificador del análisis de requerimientos del equipo (OFY-NN). En el código y en Swagger el mismo requerimiento aparece como RF-NN, con el mismo número; por ejemplo, el RF-40 de los comentarios es el OFY-40 del análisis.

ID	Funcionalidad	Módulo	Qué hace
OFY-50	Iniciar sesión	Seguridad	Valida credenciales con BCrypt y entrega un JWT con el rol real de la base de datos.
OFY-55	Alternar rol	Seguridad	Emite un token nuevo para pasar de modo Contratante a modo Profesional.
OFY-49	Registrar usuario	Identidad	Crea la cuenta, cifra la contraseña, crea el perfil de contratante y acredita referidos.
OFY-40	Verificar correo	Identidad	Marca el correo como verificado; es requisito para crear el perfil profesional.
OFY-52 · OFY-57	Consultar y editar perfiles	Identidad	Lectura y edición básica de los perfiles de trabajador y contratante.
OFY-58	Eliminar cuenta	Identidad	Baja lógica que conserva el historial.
OFY-59	Foto de perfil	Identidad	Carga de imagen JPG/PNG de hasta 5 MB.
OFY-75	Recuperar cuenta	Identidad	Inicia la recuperación sin revelar si el correo existe.
OFY-01	Crear perfil profesional	Perfil	Solo para cuentas TRABAJADOR activas y verificadas; un perfil por usuario.
OFY-02 · OFY-03	Oficio principal y secundarios	Perfil	Un oficio principal y hasta 5 secundarios, todos activos en el catálogo.
OFY-04 · OFY-05	Zona y tarifa	Perfil	Zona de cobertura y tarifa por hora en COP.
OFY-06	Disponibilidad semanal	Perfil	Hasta 28 franjas sin solapes.
OFY-07 · OFY-09	Portafolio y detalles	Perfil	Hasta 20 fotos en MongoDB y texto libre del oficio.
OFY-30 · OFY-31	Disponible ahora	Perfil	Interruptor de urgencias; exige zona de cobertura.
OFY-60	Métodos de pago	Perfil	Efectivo, Nequi o Daviplata.
OFY-65	Perfil público	Perfil	Perfil completo con portafolio.
OFY-66 · OFY-67	Especializaciones	Perfil	Especialidades por oficio guardadas en MongoDB.
OFY-42	Verificación de identidad	Confianza	Documento, antecedentes y foto; pasa a PENDIENTE_REVISION al completar los tres.
OFY-77	Catálogo de oficios	Descubrimiento	18 oficios en 6 categorías cargados por el DataSeeder.
OFY-11 a OFY-18 · OFY-32	Búsqueda de trabajadores	Descubrimiento	Texto libre, oficio, categoría, zona, tarifa, día y hora, disponibilidad, calificación y orden por reputación o distancia.
OFY-19 a OFY-21 · OFY-23 · OFY-26 · OFY-27 · OFY-29 · OFY-76 · OFY-78	Ciclo de vida de la solicitud	Servicios	Crear, enviar, aceptar, rechazar, iniciar, completar y cancelar con el patrón State.
OFY-25 · OFY-61	Notificación de cambios	Servicios	Evento de dominio, listener y estrategia de canal (push, SMS, correo).
OFY-51	Mensajería por solicitud	Servicios	Chat REST guardado en MongoDB.
OFY-08 · OFY-44 · OFY-47 · OFY-72	Historial y estadísticas	Servicios	Historial, conteos por estado y tablero para cada rol.
OFY-34 a OFY-38 · OFY-56 · OFY-71	Reseñas mutuas	Confianza	Calificación de 1 a 5 solo sobre solicitudes COMPLETADAS, promedio recalculado y reporte de reseñas.
OFY-64 · OFY-73	Reportar usuario	Confianza	Reporte con motivo y evidencia, y consulta de su estado.
OFY-54	Panel de moderación	Confianza	Listado de reportes y pausa de cuentas, solo ADMIN.
OFY-68 · OFY-69	Referidos y embajador	Comunidad	Invitación por correo; con 5 registros se otorga la insignia de embajador.

3.1 Trazabilidad con el análisis de requerimientos

El análisis de requerimientos tiene 78 requerimientos. Al cruzarlo con los controladores, 64 quedan cubiertos por al menos un endpoint (el ID aparece en la sección 5 junto a cada uno). Los 14 restantes están en la tabla siguiente, con lo que hay hoy en el código. No los presentamos como hechos: también aparecen como la limitación L14.

ID	Requerimiento	Estado en el backend	Qué hay hoy
OFY-22	Identificar trabajadores cercanos compatibles	Parcial	La cadena BarrioExacto → BarriosCercanos → Ciudad está implementada, probada e inyectada en `BusquedaTrabajadorServiceImpl`, que la usa en `buscarTrabajadores(zona, …)`. El endpoint público llama `buscar(criterios)`, que usa solo las `Specification`, así que por HTTP la cadena no se alcanza (L9).
OFY-70	Ampliar la zona cuando no hay resultados	Parcial	La ampliación barrio → barrios cercanos → ciudad es justamente esa cadena; existe en el servicio, pero ningún endpoint pasa por `buscarTrabajadores(zona, …)`.
OFY-43	Insignia de identidad verificada tras la aprobación	Parcial	`actualizarEstadoVerificacion` existe en el servicio, pero ningún endpoint lo llama para aprobar, e `IdentidadVerificadaDecorator` no se aplica al construir el perfil.
OFY-41	Habilitar la verificación después de 10 trabajos	En proceso	La verificación se puede iniciar sin contar trabajos completados; no hay ninguna regla con ese umbral.
OFY-39	Pausar al trabajador con menos de tres estrellas	Parcial	Al calificar, si el trabajador tiene 3 o más reseñas y promedio menor a 3,0 se apaga `disponibleAhora`. El análisis pide mirar las últimas cinco reseñas y pausar la cuenta.
OFY-74	Consultar trabajos completados por oficio	Parcial	`ReputacionPorOficioDTO.totalResenas` cuenta todas las reseñas del trabajador, no las del oficio (L6).
OFY-24	Enviar una solicitud a varios trabajadores	En proceso	`/enviar` recibe un único `trabajadorId`.
OFY-28	Expirar la solicitud a los 30 minutos	En proceso	No hay tarea programada (`@Scheduled`) ni estado EXPIRADA.
OFY-33	Disponibilidad en tiempo real	En proceso	No hay WebSocket ni SSE; el cliente ve el cambio cuando vuelve a consultar el perfil o la búsqueda.
OFY-45	Ingresos mensuales referenciales	En proceso	`EstadisticasResponseDTO` solo cuenta solicitudes por estado.
OFY-46	Servicios más solicitados	En proceso	Igual que OFY-45: no hay agregación por oficio.
OFY-62	Evidencia fotográfica del trabajo terminado	En proceso	`/completar` no recibe archivos.
OFY-63	Ubicación geoespacial del dispositivo	En proceso	La zona se guarda como texto y la distancia la estima `MockMapboxAdapter`.
OFY-48	Volver a contactar desde el historial	En el frontend	No necesita endpoint propio: el front toma el trabajador del historial y llama `POST /api/v1/solicitudes`.

3.2 Casos de uso

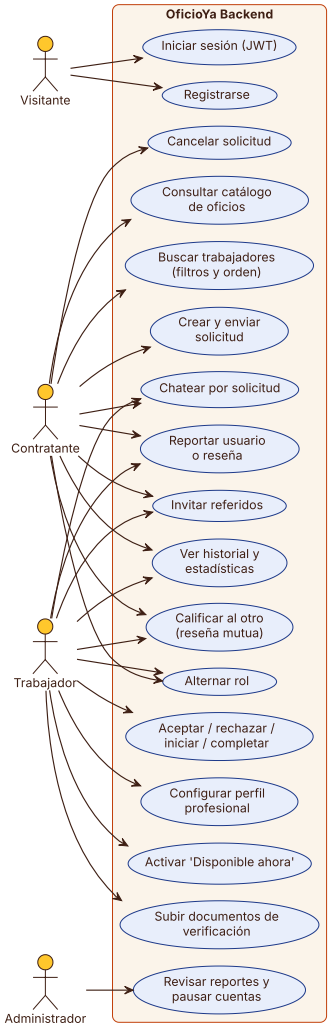


Figura 2. Casos de uso por actor.

El visitante solo puede registrarse e iniciar sesión. El contratante descubre, pide y califica; el trabajador configura su oferta y mueve la solicitud por sus estados; el administrador modera. Varias funciones son compartidas por los dos roles de negocio (chat, reseñas, reportes, referidos, historial) porque una misma persona puede ser contratante y trabajador.

4. Diagramas de arquitectura

4.1 Arquitectura en capas

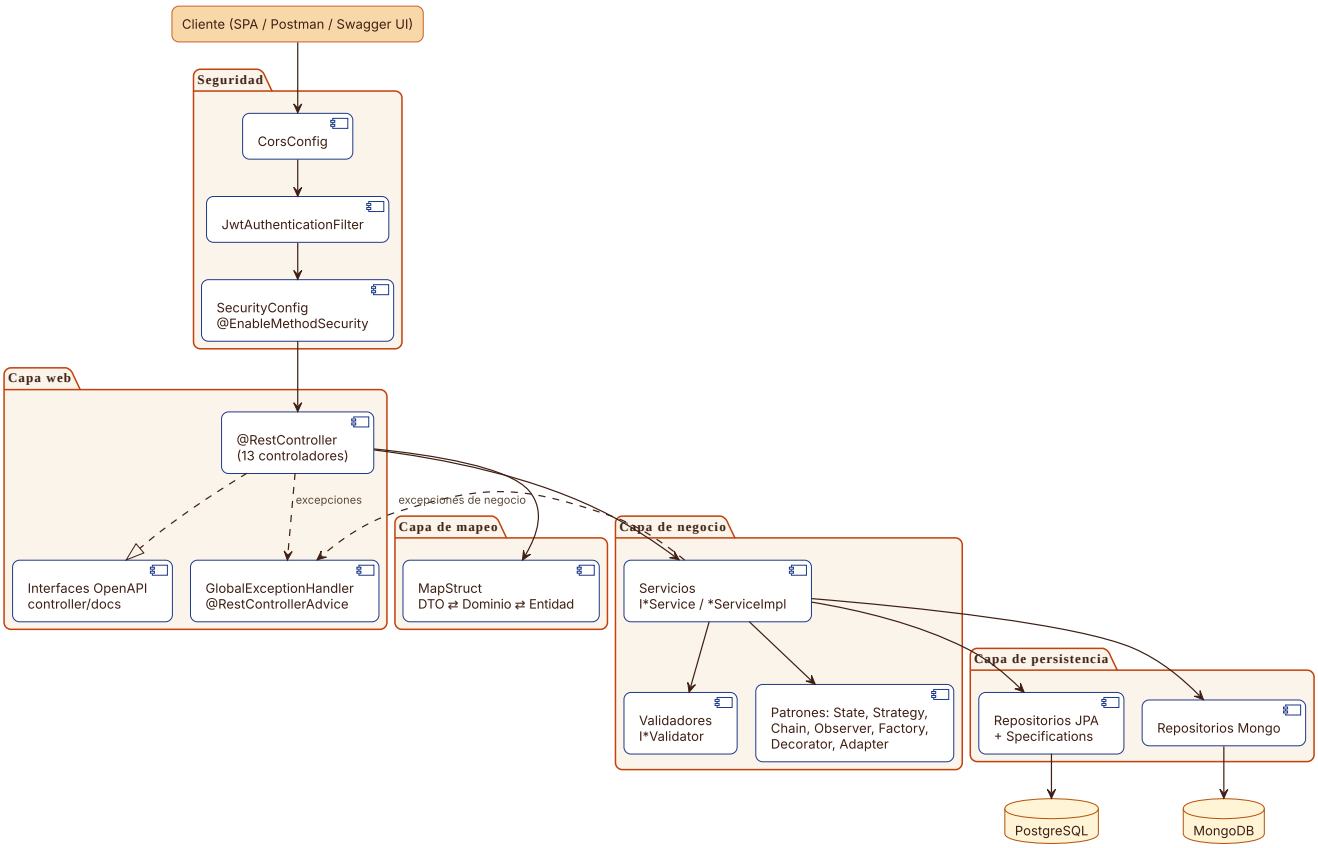


Figura 3. Capas del backend y recorrido de una petición.

Toda petición cruza primero la configuración CORS y el `JwtAuthenticationFilter`, que reconstruye la identidad a partir del token. Luego Spring Security decide si la ruta es pública o requiere autenticación, y `@PreAuthorize` revisa el rol justo antes de entrar al método del controlador. El controlador solo traduce HTTP: valida el DTO con `@Valid`, lo convierte con MapStruct y llama al servicio. El servicio orquesta la regla de negocio apoyado en validadores dedicados (`IUsuarioValidator`, `IPerfilTrabajadorValidator`, `ISolicitudValidator`, `IBusquedaValidator`) y en los patrones de diseño, y persiste a través de repositorios. Si algo falla en cualquier capa, la excepción sube hasta `GlobalExceptionHandler`, que la convierte en un `ErrorResponseDTO` con el código HTTP correcto.

Las anotaciones de OpenAPI viven en interfaces separadas (`controller/docs/*Api`) que los controladores implementan. Así el controlador queda limpio y la documentación de Swagger se mantiene en un solo lugar.

4.2 Modelo entidad-relación (PostgreSQL)

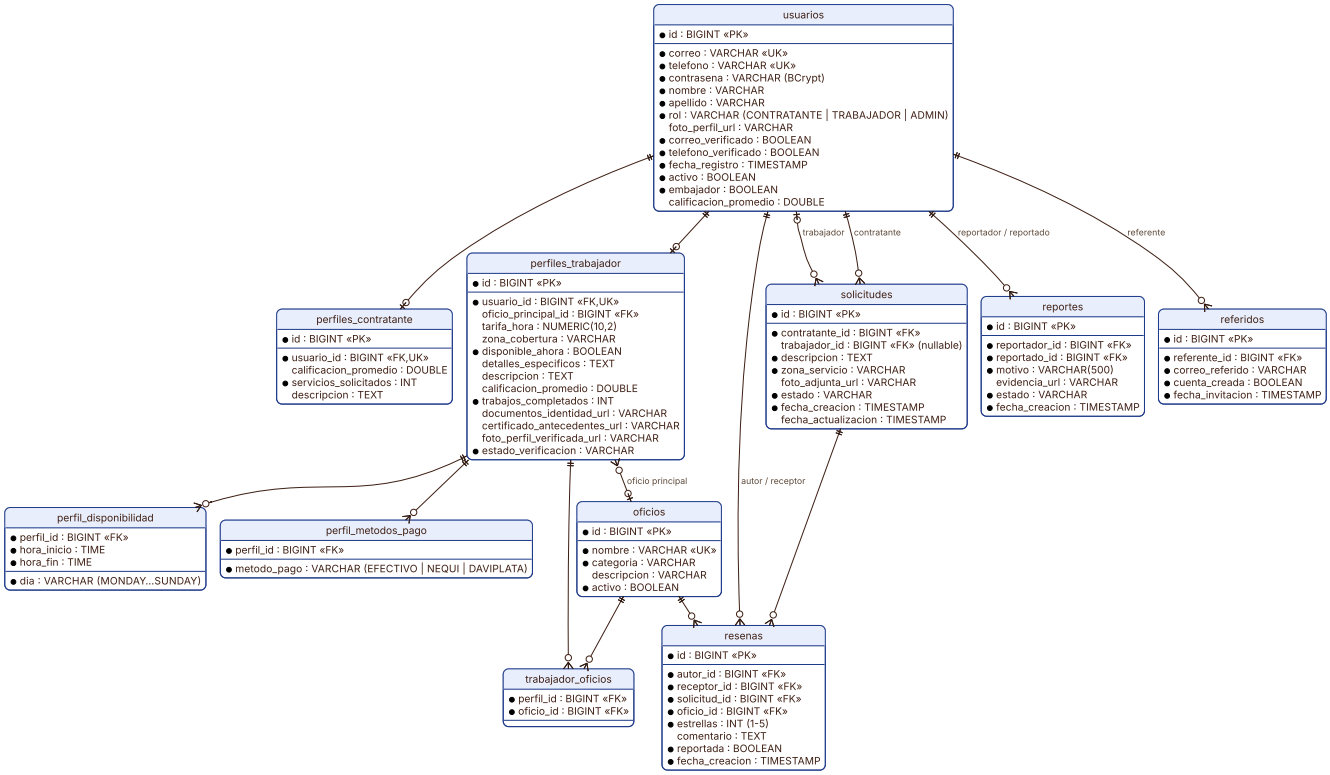


Figura 4. Modelo entidad-relación generado a partir de las entidades JPA.

Cómo leerlo. `usuarios` está en el centro. Cada cuenta puede tener a lo sumo un perfil de contratante y a lo sumo un perfil de trabajador (relaciones 1 a 0..1 con `usuario_id` único). El perfil de trabajador tiene un oficio principal y una lista de oficios secundarios resuelta con la tabla intermedia `trabajador_oficios`. Su horario y sus medios de pago son colecciones de valores (`@ElementCollection`) en `perfil_disponibilidad` y `perfil_metodos_pago` .

`solicitudes` siempre tiene contratante y su trabajador puede ser nulo, lo que deja la puerta abierta al modelo en el que un trabajador cercano toma la solicitud. Las `resenas` amarran autor, receptor, solicitud y oficio, lo que permite la calificación mutua y la reputación por oficio. `reportes` y `referidos` completan la parte de comunidad. El esquema lo crea Hibernate (`ddl-auto=update`).

Tabla	Qué guarda	Reglas clave
usuarios	Cuenta, credenciales y estado	correo y teléfono únicos · contraseña BCrypt · baja lógica con `activo`
perfiles_contratante	Datos del modo Contratante	1 por usuario · se crea al registrarse como contratante
perfiles_trabajador	Oferta profesional	1 por usuario · tarifa NUMERIC(10,2) · estado de verificación
oficios	Catálogo	nombre único · 18 registros semilla
trabajador_oficios	Oficios secundarios	N a N entre perfil y oficio
perfil_disponibilidad	Franjas semanales	día + hora inicio + hora fin
perfil_metodos_pago	Medios de pago	EFECTIVO · NEQUI · DAVIPLATA
solicitudes	Servicios pedidos	estado del enum EstadoSolicitud · trabajador opcional
resenas	Calificaciones	1 a 5 estrellas · bandera `reportada`
reportes	Denuncias	motivo de hasta 500 caracteres · estado del reporte
referidos	Invitaciones	correo invitado · cuenta creada

4.3 Modelo documental (MongoDB)

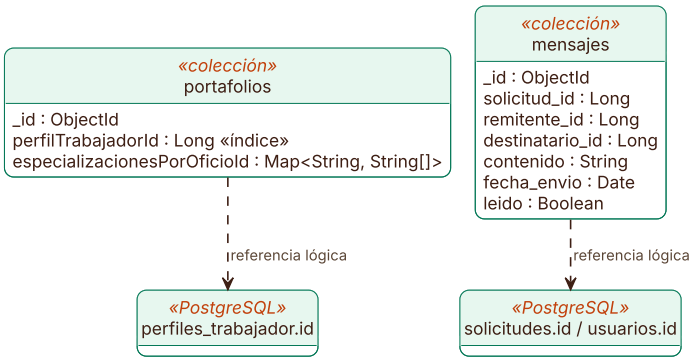


Figura 5. Colecciones de MongoDB y su referencia lógica a PostgreSQL.

La colección `portafolios` guarda, por perfil, la lista de URLs de fotos y un mapa de especializaciones por oficio. El campo `perfilTrabajadorId` tiene índice y es la referencia lógica al perfil en PostgreSQL. La colección `mensajes` guarda cada mensaje del chat con su solicitud, remitente, destinatario, fecha y si fue leído. Como no hay transacciones entre motores, la actualización del portafolio guarda primero en PostgreSQL y, si MongoDB falla, lanza una excepción para que `@Transactional` revierta la parte relacional.

4.4 Diagramas de clases

Los diagramas de esta sección se generaron leyendo directamente el código de `develop` (commit `890b8b8`): cada clase muestra sus atributos y sus métodos públicos con la misma firma que tiene en Java, y las relaciones salen de las anotaciones (`@OneToMany` , `@ManyToOne` , `@ElementCollection`), de la herencia, de las interfaces implementadas y de los campos inyectados. En total aparecen 149 de los 193 archivos del backend; los 37 DTO no se dibujan porque sus campos están completos en la sección 6. Como el modelo no cabe en una sola figura legible, lo dividimos por capa y por módulo. Las figuras son vectoriales: se pueden ampliar en el PDF sin perder nitidez.

Dominio y patrón State

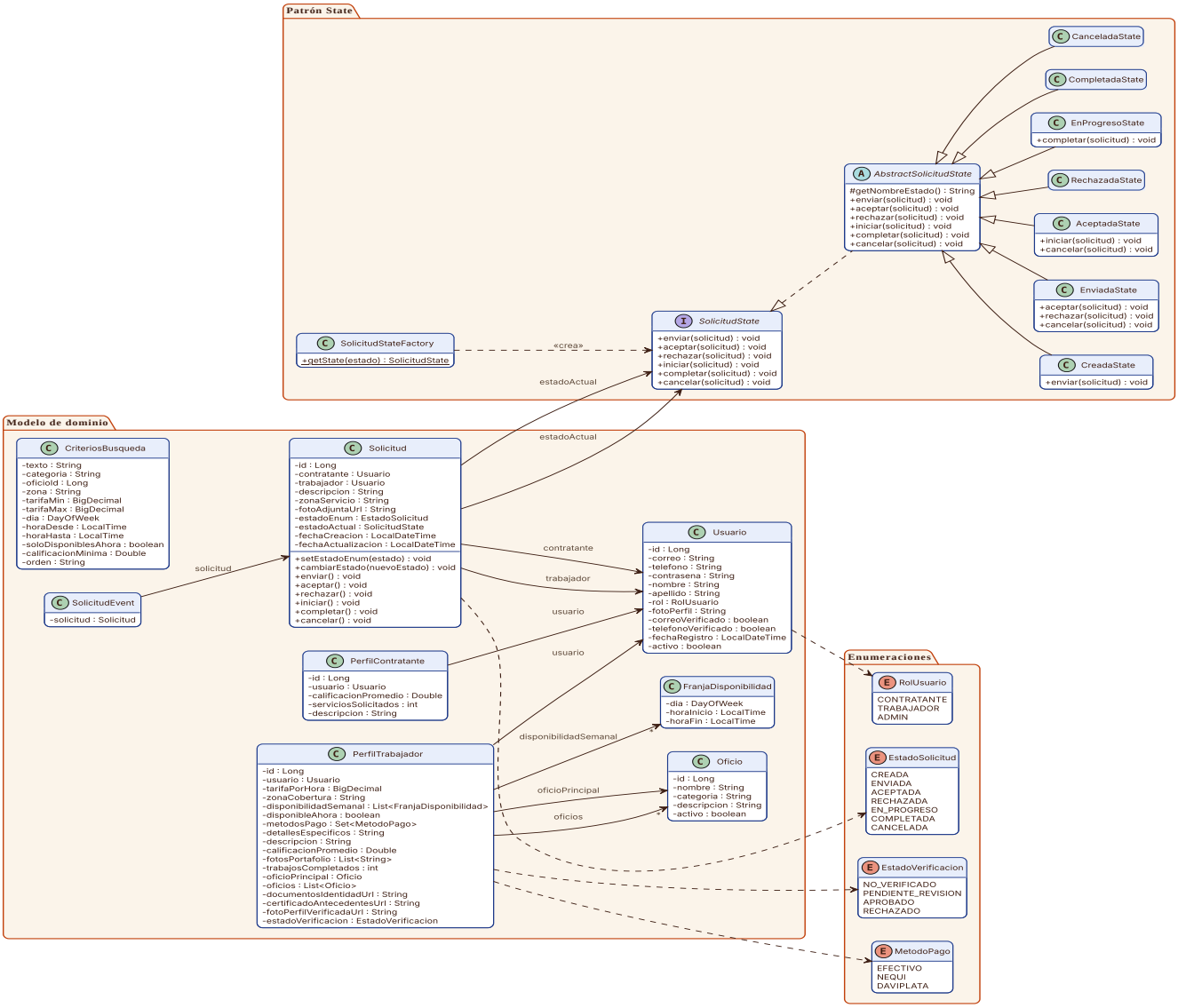


Figura 6a. Modelo de dominio, enumeraciones y jerarquía de estados de la solicitud.

El dominio está separado de las entidades JPA: los servicios trabajan con `Usuario`, `PerfilTrabajador` o `Solicitud`, y los mappers los convierten a entidades solo al llegar a la persistencia. `Solicitud` no guarda su comportamiento en `if`: delega cada acción en un `SolicitudState`, que `SolicitudStateFactory` crea a partir del enum `EstadoSolicitud`. `CriteriosBusqueda` y `SolicitudEvent` son objetos de valor que viajan entre capas.

Persistencia

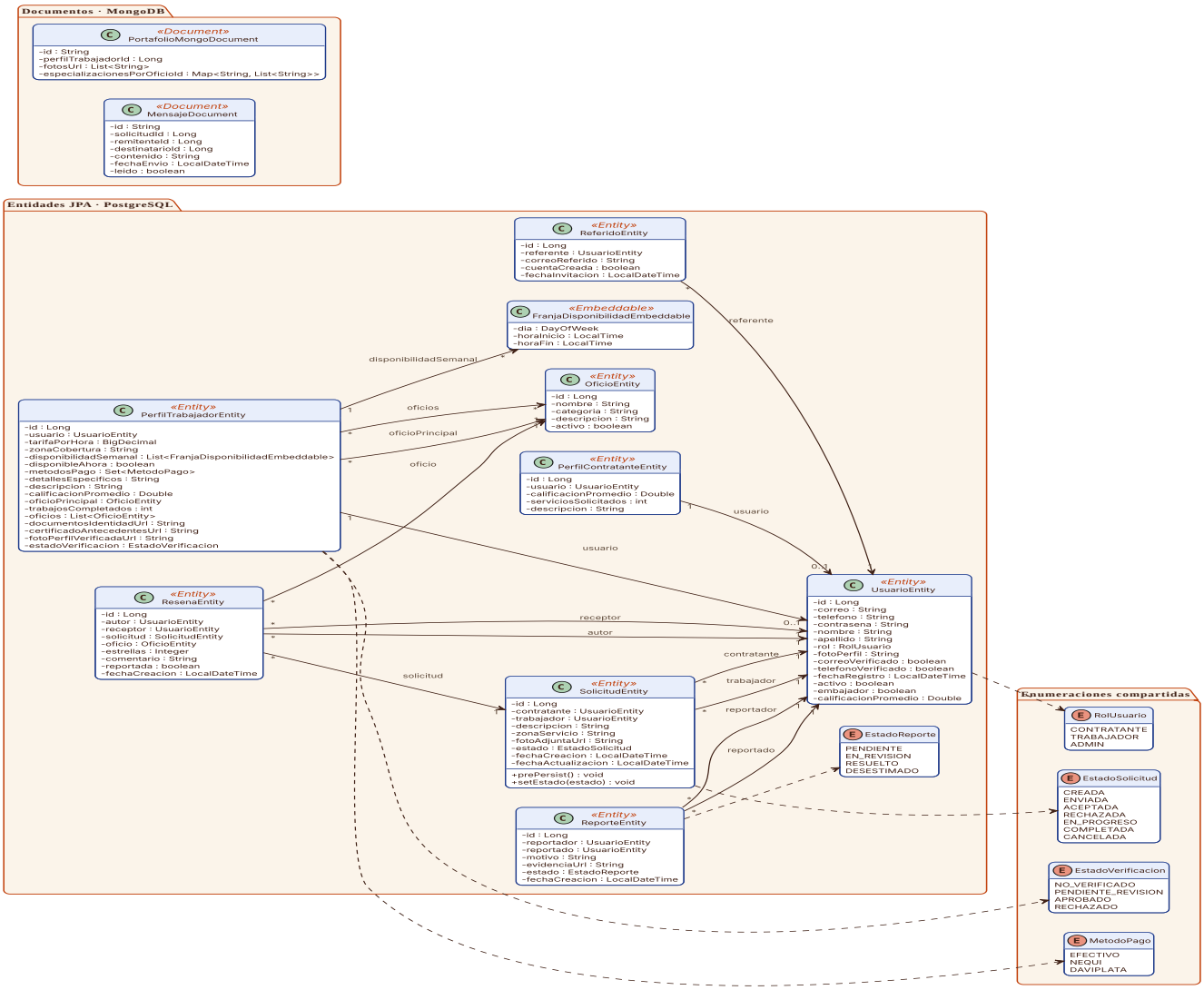


Figura 6b. Entidades JPA con sus multiplicidades, enumeraciones y documentos de MongoDB.

Es la misma información del modelo entidad-relación, pero vista como clases: cada entidad con sus campos y las relaciones con su multiplicidad real. `FranjaDisponibilidadEmbeddable` es un valor embebido del perfil, y los dos documentos de MongoDB se enlazan con PostgreSQL solo por identificador.

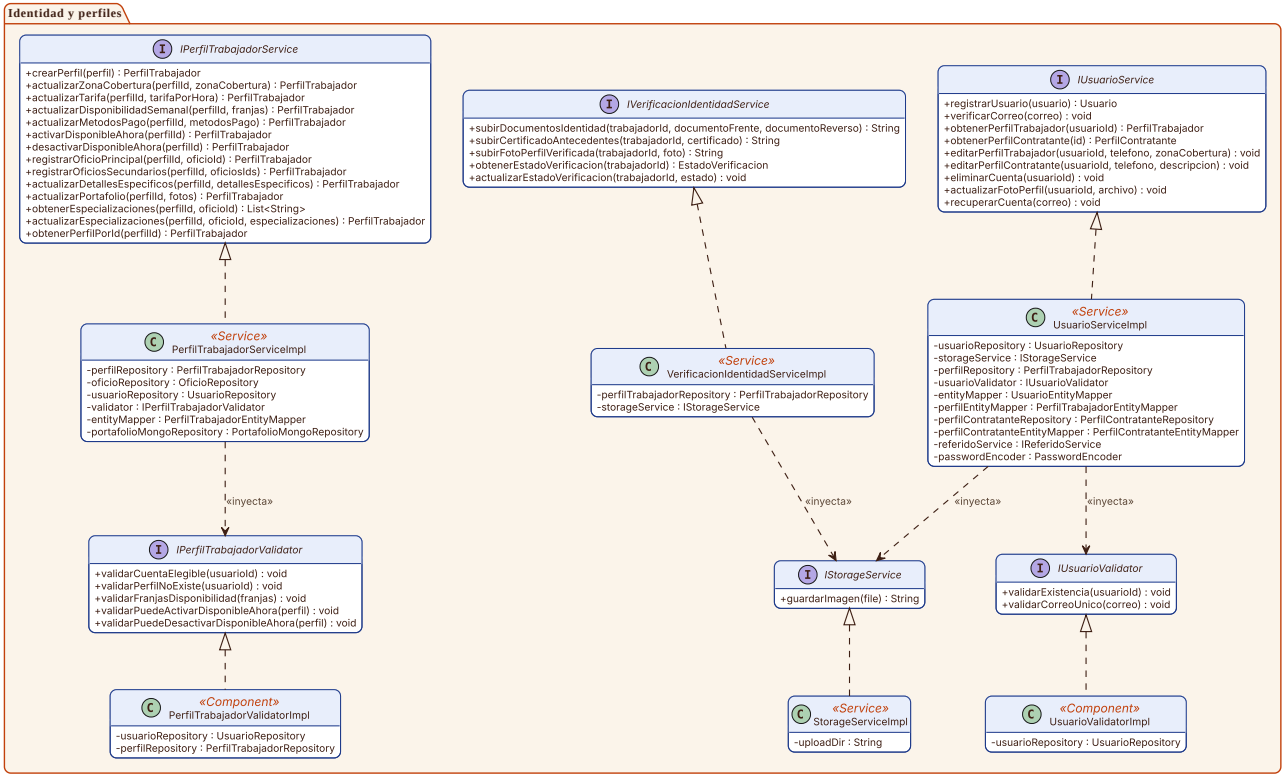


Figura 6c. Servicios de identidad y perfiles con sus validadores y el almacenamiento de archivos.

Cada servicio se declara como interfaz y tiene una sola implementación. Las reglas que pueden fallar viven en validadores aparte (`IUsuarioValidator`, `IPerfilTrabajadorValidator`), y la subida de fotos y documentos pasa siempre por `IStorageService`.

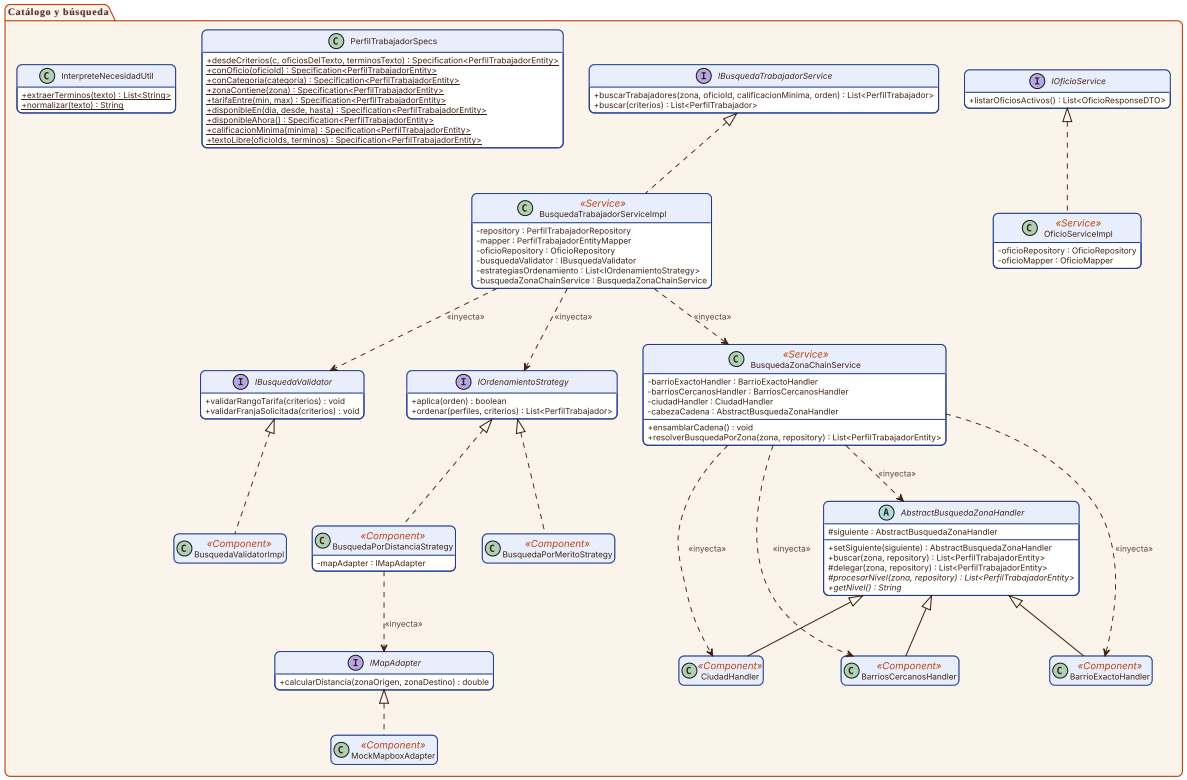


Figura 6d. Catálogo de oficios y búsqueda: Strategy, Adapter, Chain of Responsibility y Specification.

`BusquedaTrabajadorServiceImpl` recibe las dos estrategias de orden, el adaptador de mapas y la cadena de zonas. `PerfilTrabajadorSpecs` arma los filtros y `InterpreteNecesidadUtil` traduce texto libre a oficios.

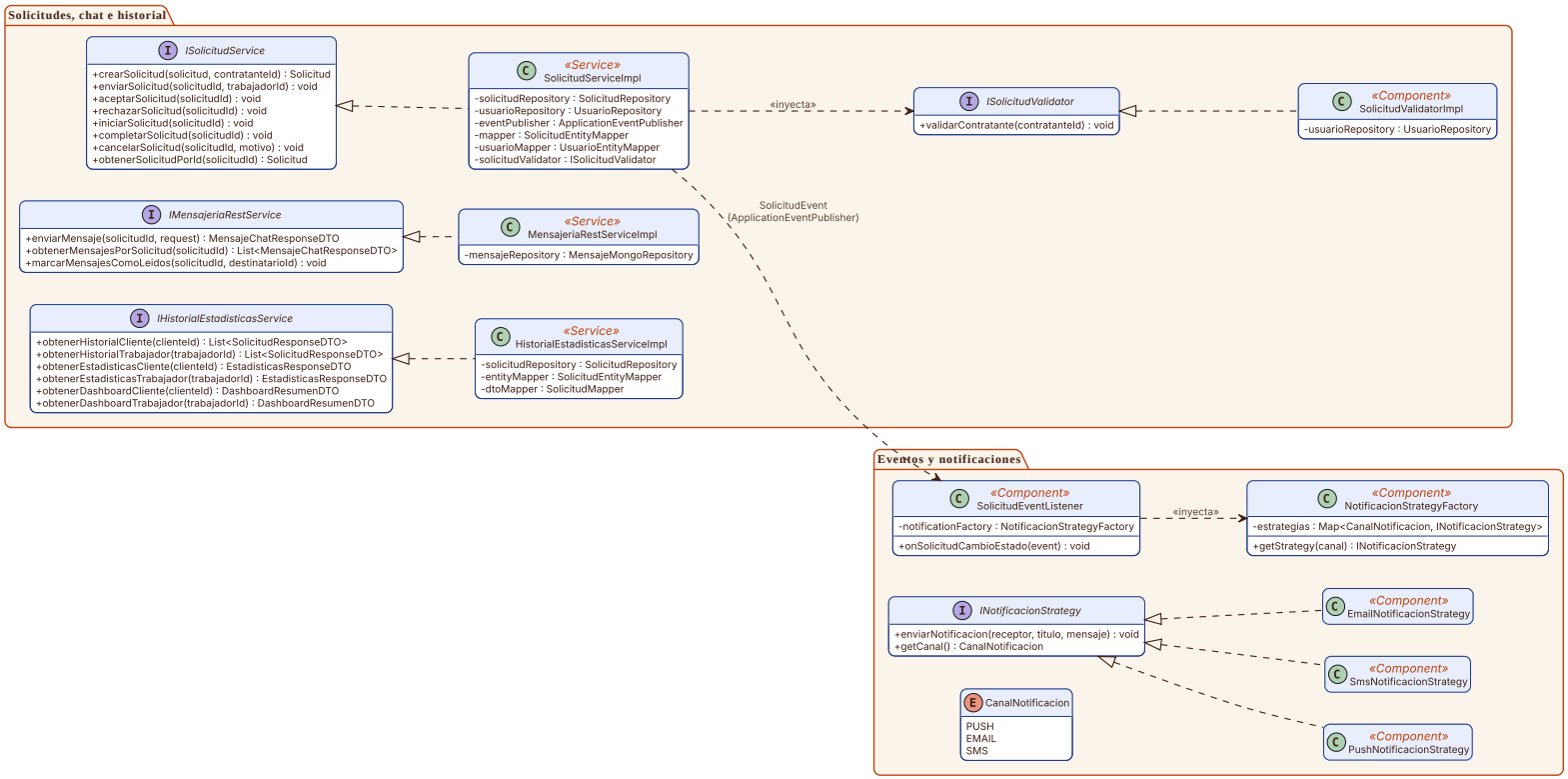


Figura 6e. Solicitudes, mensajería e historial, con el evento y las estrategias de notificación.

`SolicitudServiceImpl` publica un `SolicitudEvent` y no conoce las notificaciones: `SolicitudEventListener` lo escucha y pide a `NotificacionStrategyFactory` la estrategia del canal.

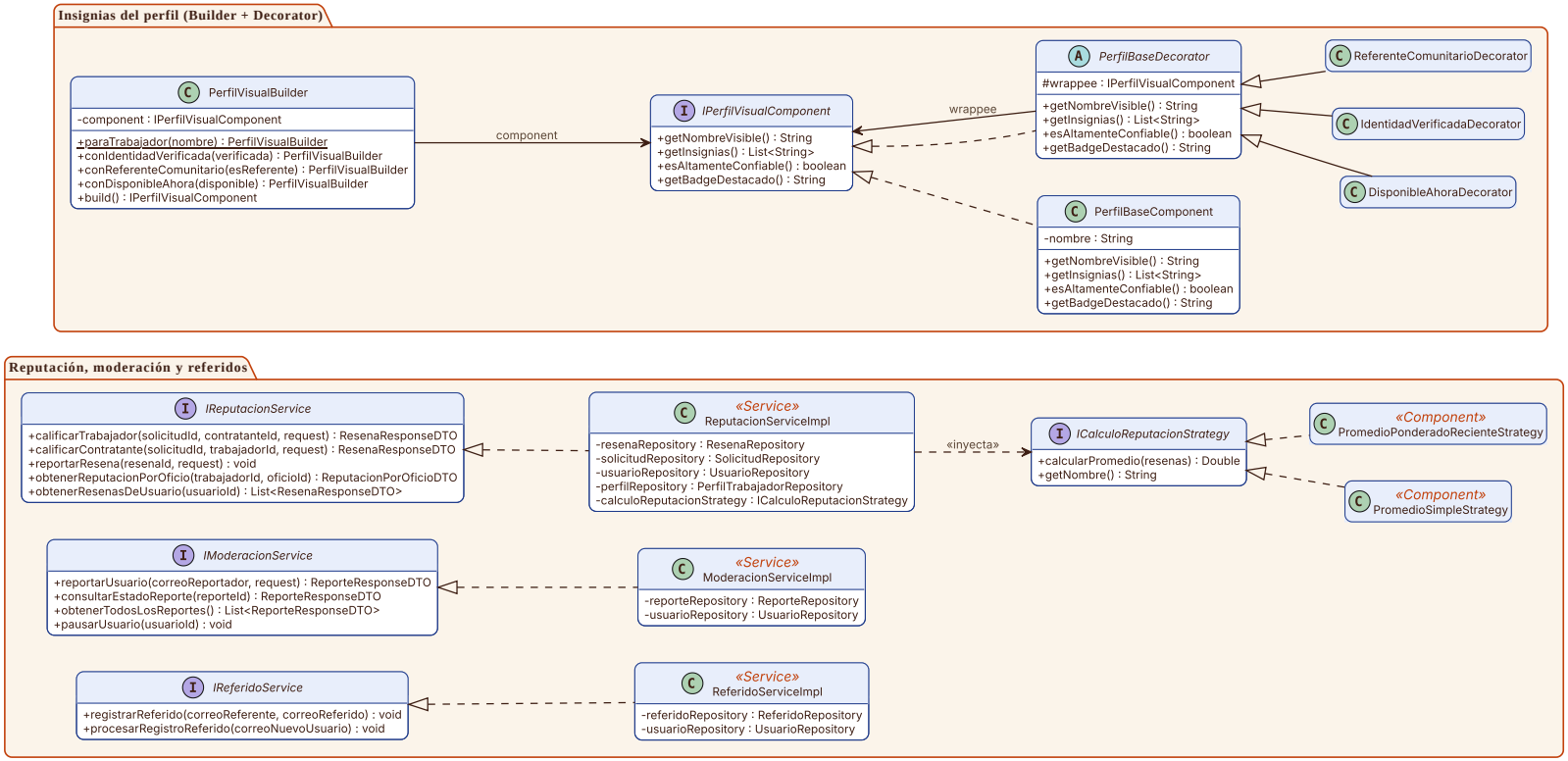


Figura 6f. Reputación, moderación y referidos, con el Builder y los decoradores de insignias.

El cálculo del promedio es una estrategia intercambiable. Las insignias del perfil se apilan con decoradores sobre `PerfilBaseComponent`, y `PerfilVisualBuilder` decide cuáles aplicar.

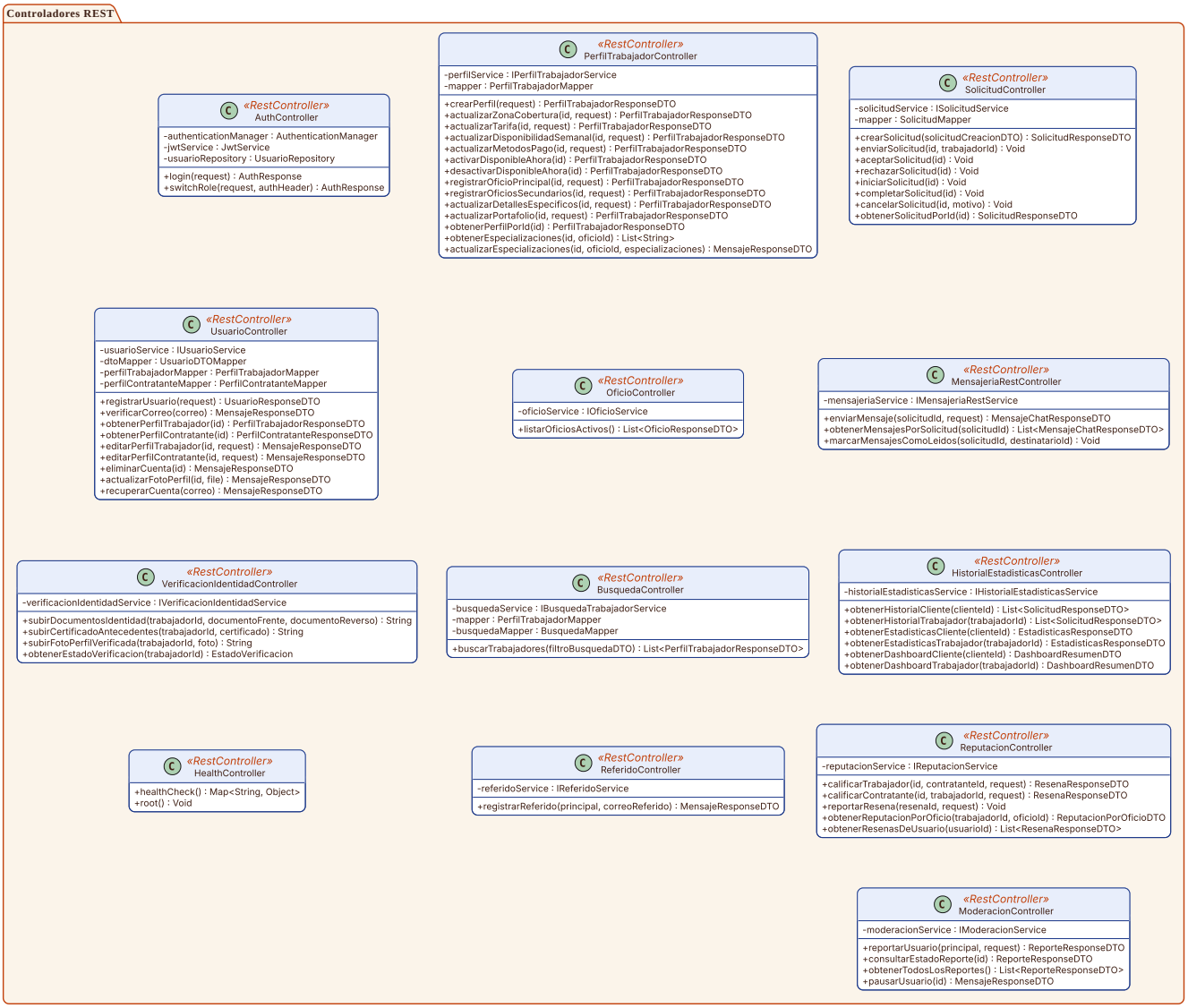


Figura 6g. Los 13 controladores REST con sus endpoints y el servicio que usa cada uno.

Los controladores solo traducen HTTP: cada método corresponde a un endpoint de la sección 5 y delega en una interfaz de servicio. Las anotaciones de Swagger están en las interfaces `controller/docs/*Api` que cada controlador implementa.

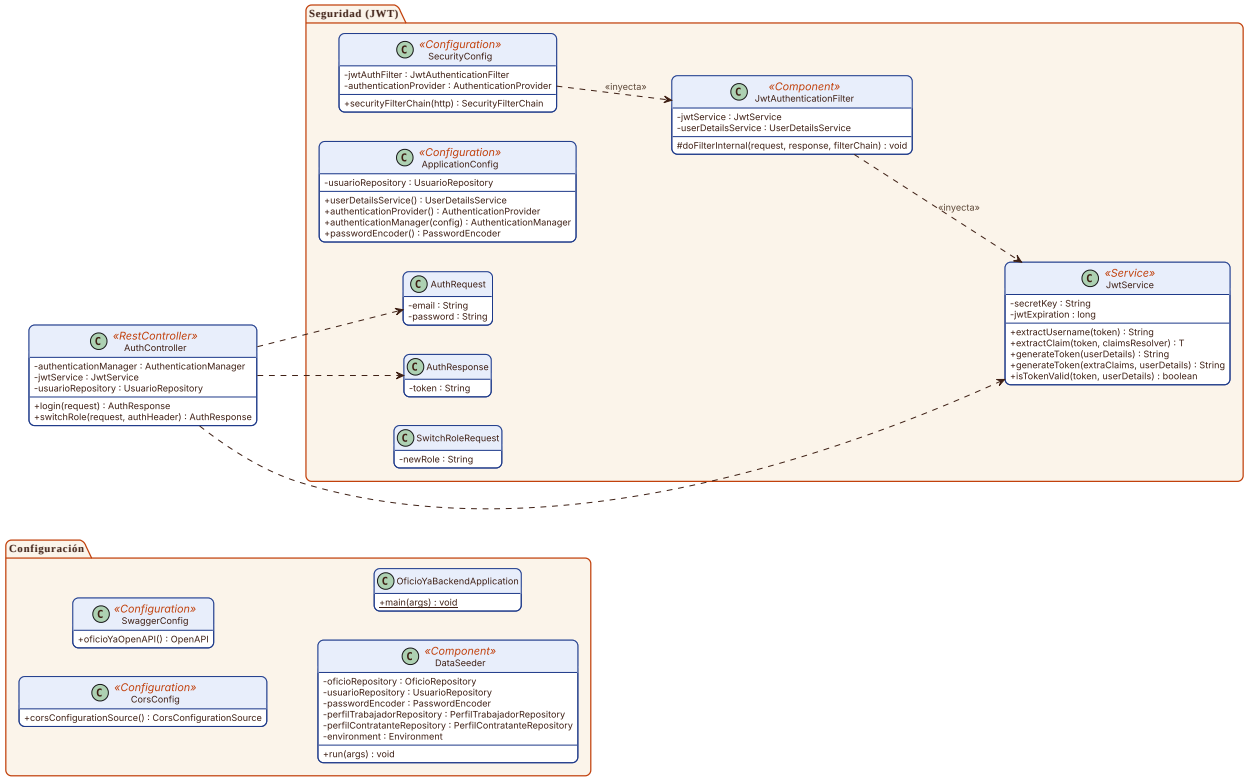


Figura 6h. Seguridad JWT, configuración y arranque de la aplicación.

`SecurityConfig` registra `JwtAuthenticationFilter` antes del filtro de usuario y contraseña; el filtro usa `JwtService` para leer el token y `ApplicationConfig` aporta el `UserDetailsService`, el `PasswordEncoder` BCrypt y el `AuthenticationManager`. `DataSeeder` carga el catálogo y los usuarios de demostración.

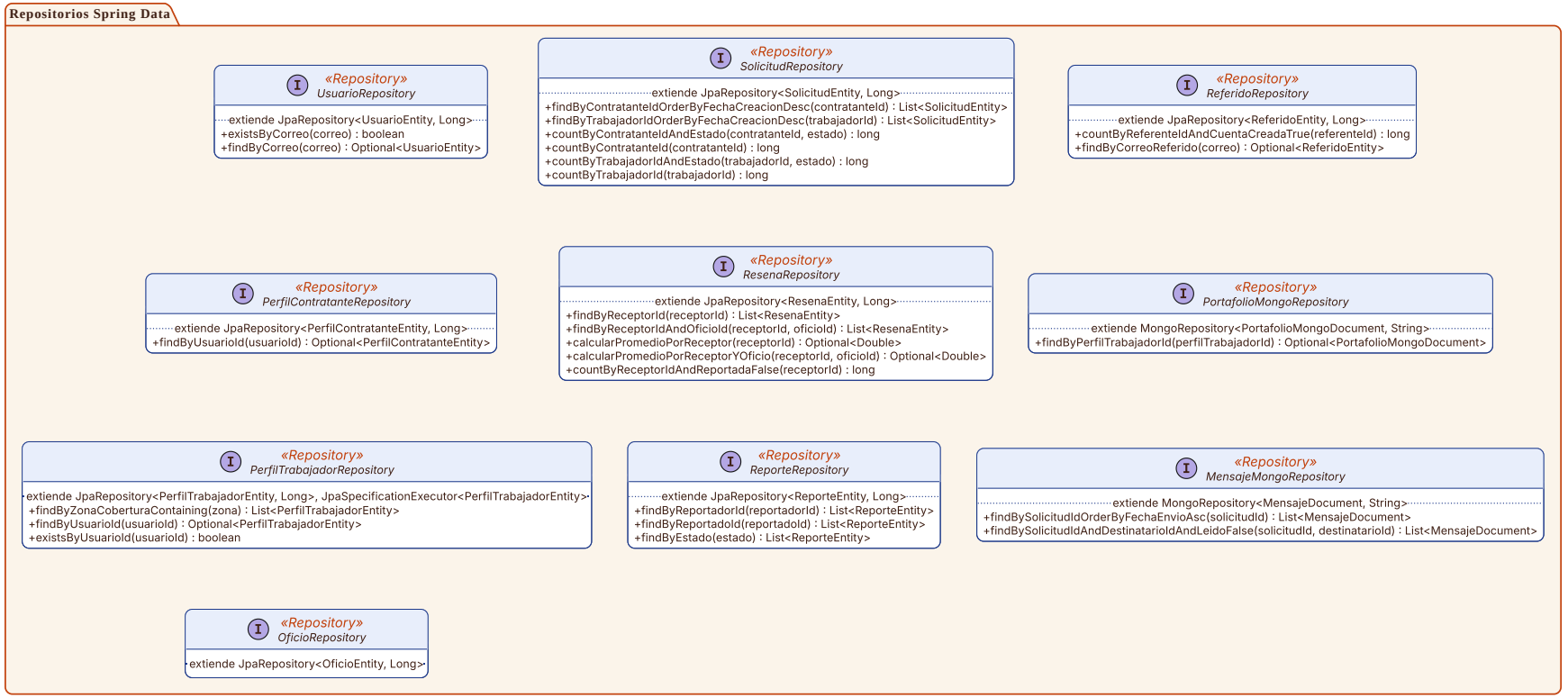


Figura 6i. Repositorios de Spring Data JPA y MongoDB con sus consultas derivadas.

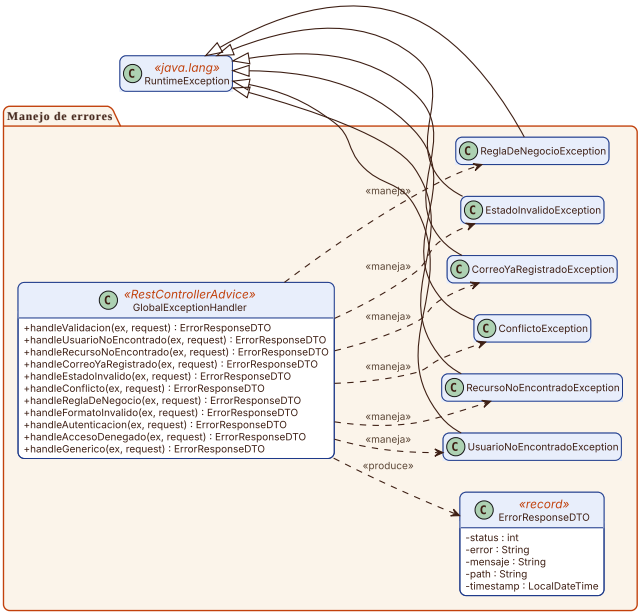


Figura 6j. Excepciones de negocio y su manejador global.

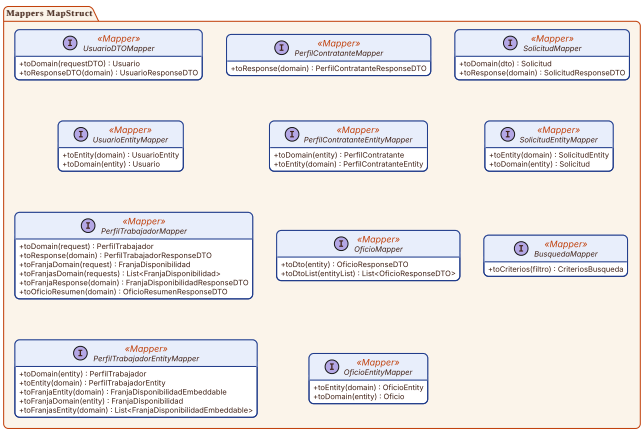


Figura 6k. Mappers de MapStruct entre DTO, dominio y entidad.

Las seis excepciones de negocio extienden `RuntimeException` y `GlobalExceptionHandler` las traduce a un `ErrorResponseDTO` (sección 7). Cada agregado tiene dos mappers: uno entre DTO y dominio, para los controladores, y otro entre dominio y entidad, para los servicios.

4.5 Patrones de diseño implementados

State: ciclo de vida de la solicitud

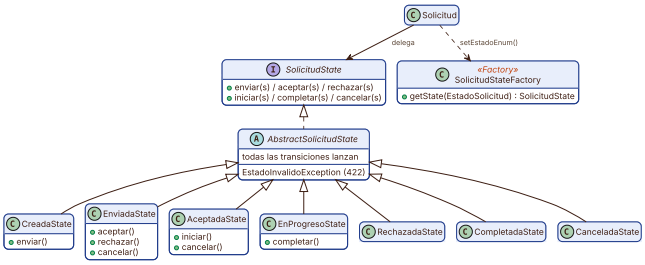


Figura 7. Jerarquía de estados.

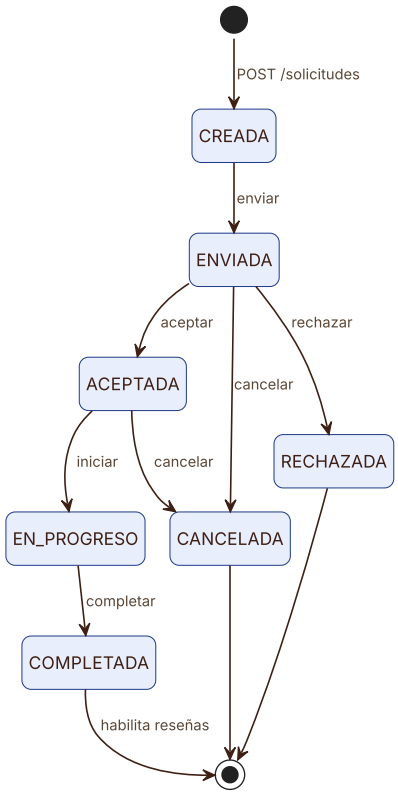


Figura 8. Transiciones legales de una solicitud.

`AbstractSolicitudState` implementa todas las acciones lanzando `EstadoInvalidoException`; cada estado concreto sobrescribe solo las transiciones que le son legales. Por eso cancelar una solicitud COMPLETADA no es un caso olvidado en un `if`: simplemente no existe, y la API responde 422 con el mensaje "No se puede CANCELAR la solicitud desde el estado COMPLETADA". `SolicitudStateFactory` reconstruye el objeto de estado cada vez que la solicitud sale de la base de datos.

Strategy, Adapter, Chain of Responsibility y Specification: búsqueda

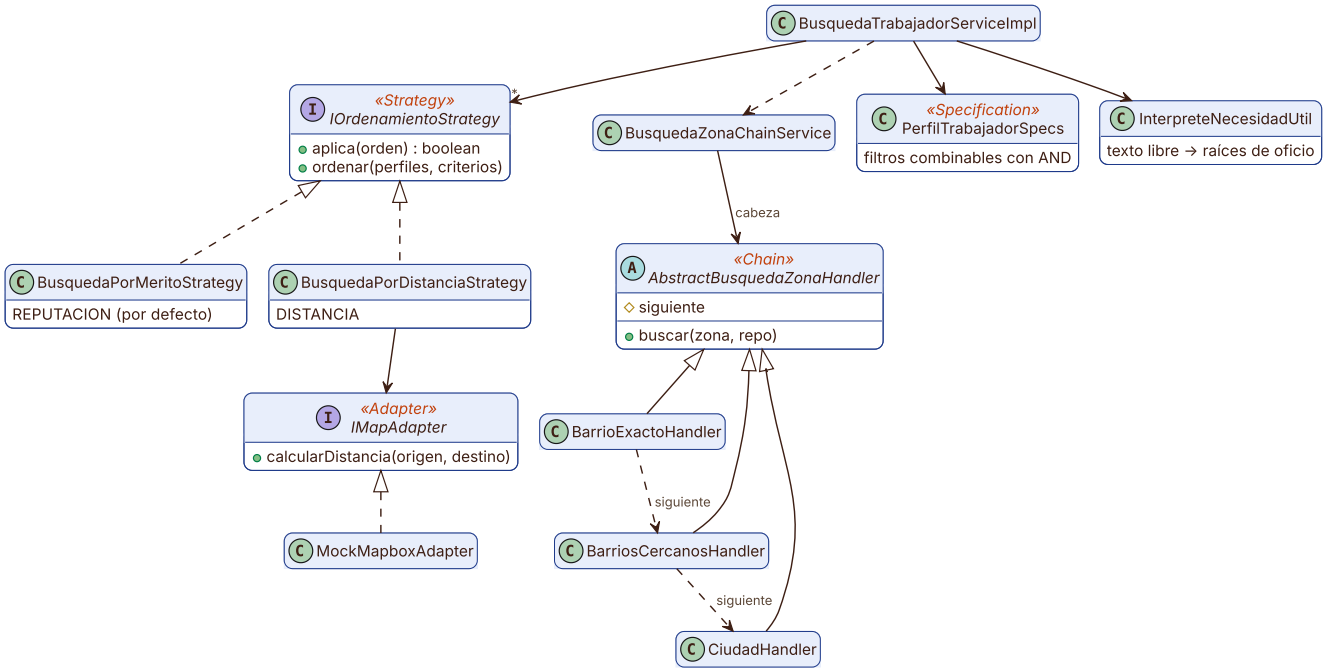


Figura 9. Patrones del módulo de búsqueda.

El orden de los resultados es una estrategia: `BusquedaPorMeritoStrategy` (por defecto, reputación y luego trabajos completados) o `BusquedaPorDistanciaStrategy`, que consulta la distancia a través del adaptador `IMapAdapter`. Los filtros se arman como `Specification` independientes que se unen con AND y se traducen a una sola consulta SQL. `InterpreteNecesidadUtil` convierte texto libre en raíces de oficio con un diccionario de sinónimos ("gotera" → plomería, "enchufe" → electricidad). La cadena `BarrioExacto → BarriosCercanos → Ciudad` amplía el radio cuando no hay resultados; está lista en el servicio y probada, y el servicio la usa en `buscarTrabajadores(zona, …)`, aunque el endpoint público llama otro método y todavía no pasa por ella (ver L9).

Strategy, Observer, Factory, Builder y Decorator: reputación, notificaciones e insignias

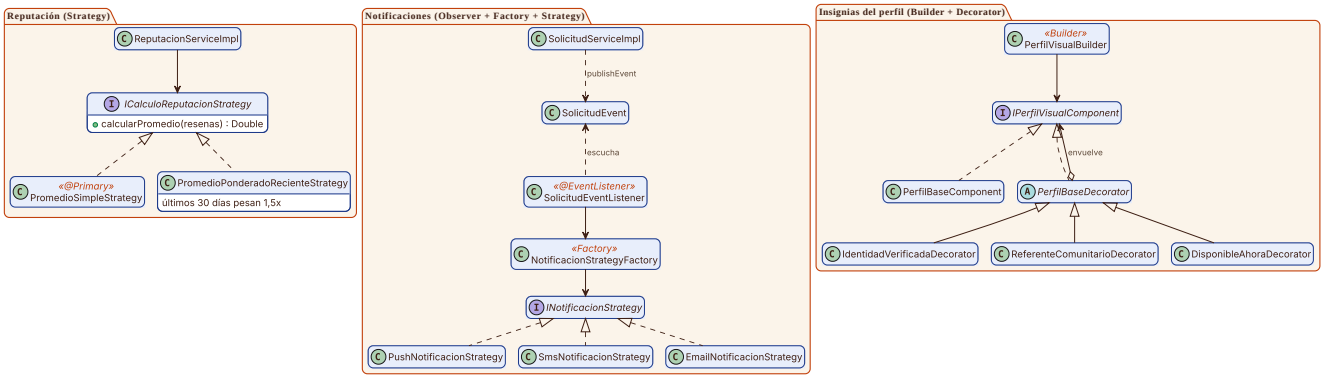


Figura 10. Patrones de reputación, notificaciones e insignias.

El promedio de reputación se calcula con `ICalculoReputacionStrategy`; hoy está activo el promedio simple (`@Primary`) y existe una variante que da 1,5 veces más peso a las reseñas de los últimos 30 días. Cuando una solicitud se envía, el servicio publica un `SolicitudEvent`; el `SolicitudEventListener` lo escucha (Observer), pide a la fábrica la estrategia del canal y envía el aviso, sin que el servicio de solicitudes sepa nada de notificaciones. Las insignias del perfil (identidad verificada, referente comunitario, disponible ahora) se apilan con decoradores que un Builder compone según el estado del trabajador.

4.6 Diagrama de componentes general

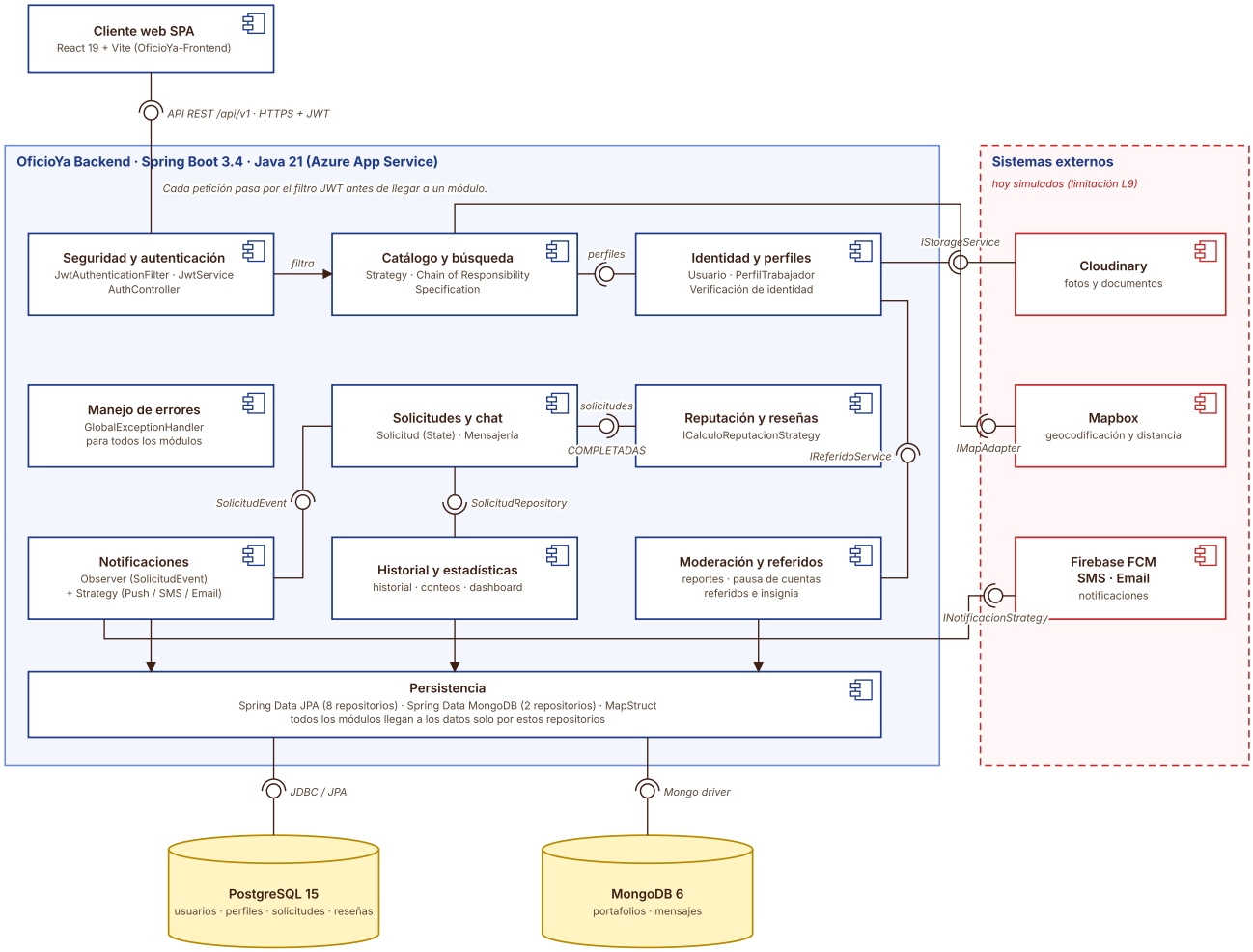


Figura 11. Componentes de alto nivel del backend en notación UML.

Seguimos la notación UML de componentes: cada caja con el ícono de componente es un módulo, el círculo (*lollipop*) es la interfaz que un módulo ofrece y el semicírculo (*socket*) es la interfaz que otro requiere. La SPA consume la única interfaz pública, la API REST `/api/v1` sobre HTTPS con JWT, y toda petición entra por el módulo de seguridad. Dentro del backend se ven los nueve módulos y las interfaces por las que se hablan: solicitudes ofrece `SolicitudRepository` al historial, reputación califica solo solicitudes COMPLETADAS, notificaciones escucha `SolicitudEvent` e identidad usa `IReferidoService` para acreditar invitaciones al registrar una cuenta. Todos llegan a los datos por la capa de persistencia, que habla con PostgreSQL por JPA y con MongoDB por su driver. A la derecha, los tres sistemas externos con la interfaz que los aísla.

4.7 Diagrama de componentes específico

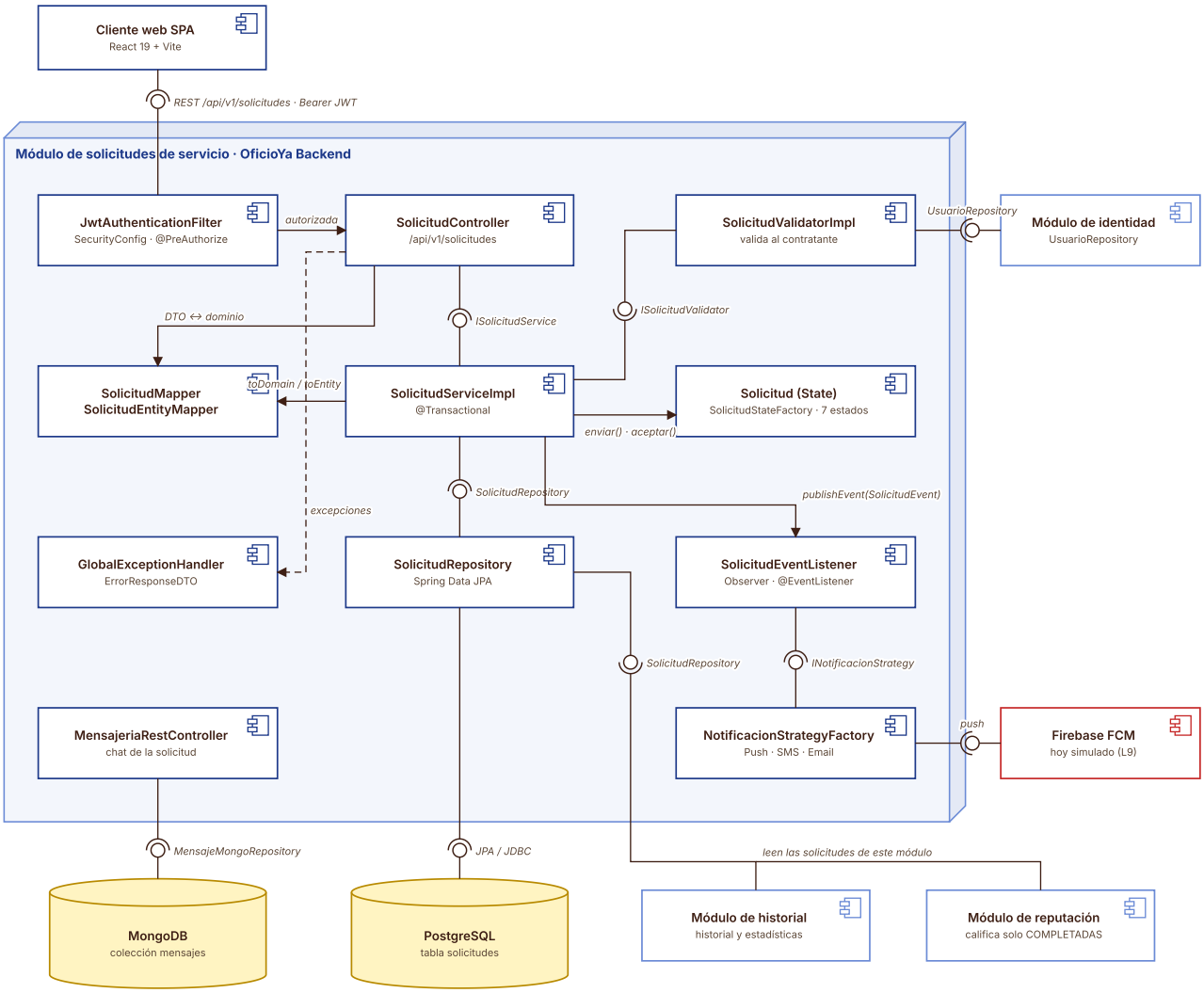


Figura 12. Componentes internos del módulo de solicitudes de servicio.

Abrimos el módulo central del producto, el de solicitudes, para ver sus componentes por dentro. La petición pasa por `JwtAuthenticationFilter` y `@PreAuthorize` antes de llegar a `SolicitudController`, que convierte el DTO con los mappers y llama a `ISolicitudService`. `SolicitudServiceImpl` valida al contratante con `ISolicitudValidator` (que a su vez requiere el `UsuarioRepository` del módulo de identidad), delega la transición en el objeto de estado de `Solicitud`, guarda con `SolicitudRepository` en PostgreSQL y publica un `SolicitudEvent`. `SolicitudEventListener` recibe el evento y, por `INotificacionStrategy`, envía el aviso al canal elegido (Firebase, hoy simulado). El chat de la solicitud vive en MongoDB a través de `MensajeriaRestController`. Los módulos de historial y de reputación no entran al servicio: leen el mismo `SolicitudRepository`. Cualquier excepción termina en `GlobalExceptionHandler`.

5. Funcionalidades expuestas (API)

5.1 Convenciones

- **URL base en la nube:** `https://oficioya-backend.azurewebsites.net` · local: `http://localhost:8082`.
- **Formato:** JSON en UTF-8, salvo las cargas de archivos (`multipart/form-data`).
- **Autenticación:** cabecera `Authorization: Bearer <token>` en todo lo que no sea público. En Swagger se usa el botón *Authorize* (esquema `bearerAuth`).
- **Fechas:** ISO-8601 (`2026-10-20T13:05:10`); horas como `HH:mm`; días con el enum de Java (`MONDAY` … `SUNDAY`).
- **Dinero:** pesos colombianos con `BigDecimal`, hasta dos decimales.
- **Errores:** siempre con el formato `ErrorResponseDTO` de la sección 6.

5.2 Resumen de los 60 endpoints

Método	Ruta	Rol requerido	RF / HU	Éxito
POST	`/auth/login`	Público	OFY-50	200 OK
POST	`/auth/switch-role`	Cualquier token válido (cabecera Authorization)	OFY-55	200 OK
GET	`/api/v1/health`	Público	Soporte técnico	200 OK
GET	`/`	Público	Soporte técnico	302 Found
POST	`/api/v1/usuarios/registro`	Público (sin token)	OFY-49	201 Created
PATCH	`/api/v1/usuarios/{correo}/verificar`	TRABAJADOR, CONTRATANTE, ADMIN	OFY-40	200 OK
GET	`/api/v1/usuarios/{id}/perfil-trabajador`	TRABAJADOR, CONTRATANTE, ADMIN	OFY-52	200 OK
GET	`/api/v1/usuarios/{id}/perfil-contratante`	TRABAJADOR, CONTRATANTE, ADMIN	OFY-53	200 OK
PUT	`/api/v1/usuarios/{id}/perfil-trabajador`	TRABAJADOR, ADMIN	OFY-57	200 OK
PUT	`/api/v1/usuarios/{id}/perfil-contratante`	CONTRATANTE, ADMIN	OFY-57	200 OK
DELETE	`/api/v1/usuarios/{id}`	TRABAJADOR, CONTRATANTE	OFY-58	200 OK
POST	`/api/v1/usuarios/{id}/foto`	TRABAJADOR, CONTRATANTE	OFY-59	200 OK
POST	`/api/v1/usuarios/recuperar-cuenta?correo=`	Público (sin token)	OFY-75	200 OK
POST	`/api/v1/perfiles-trabajador`	TRABAJADOR	OFY-01	201 Created
PUT	`/api/v1/perfiles-trabajador/{id}/oficio-principal`	TRABAJADOR	OFY-02	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/oficios-secundarios`	TRABAJADOR	OFY-03	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/zona-cobertura`	TRABAJADOR	OFY-04	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/tarifa`	TRABAJADOR	OFY-05	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/disponibilidad-semanal`	TRABAJADOR	OFY-06	200 OK
PATCH	`/api/v1/perfiles-trabajador/{id}/portafolio`	TRABAJADOR	OFY-07	200 OK
PATCH	`/api/v1/perfiles-trabajador/{id}/detalles-especificos`	TRABAJADOR	OFY-09	200 OK
PATCH	`/api/v1/perfiles-trabajador/{id}/disponible-ahora/activar`	TRABAJADOR	OFY-30	200 OK
PATCH	`/api/v1/perfiles-trabajador/{id}/disponible-ahora/desactivar`	TRABAJADOR	OFY-31	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/metodos-pago`	TRABAJADOR	OFY-60	200 OK
GET	`/api/v1/perfiles-trabajador/{id}`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-65	200 OK
GET	`/api/v1/perfiles-trabajador/{id}/oficios/{oficioId}/especializaciones`	TRABAJADOR, CONTRATANTE	OFY-66	200 OK
PUT	`/api/v1/perfiles-trabajador/{id}/oficios/{oficioId}/especializaciones`	TRABAJADOR	OFY-67	200 OK
POST	`/api/v1/trabajadores/verificacion/{trabajadorId}/documentos`	TRABAJADOR, ADMIN	OFY-42	200 OK
POST	`/api/v1/trabajadores/verificacion/{trabajadorId}/antecedentes`	TRABAJADOR, ADMIN	OFY-42	200 OK
POST	`/api/v1/trabajadores/verificacion/{trabajadorId}/foto-verificada`	TRABAJADOR, ADMIN	OFY-42	200 OK
GET	`/api/v1/trabajadores/verificacion/{trabajadorId}/estado`	TRABAJADOR, ADMIN	OFY-42	200 OK
GET	`/api/v1/oficios`	Público (sin token)	OFY-77	200 OK
GET	`/api/v1/busqueda/trabajadores`	Público (sin token)	OFY-11 a OFY-18, OFY-32	200 OK
POST	`/api/v1/solicitudes`	CONTRATANTE, ADMIN	OFY-19 · OFY-20 · OFY-21	201 Created
POST	`/api/v1/solicitudes/{id}/enviar?trabajadorId=`	CONTRATANTE, ADMIN	OFY-23 · OFY-25	200 OK
PATCH	`/api/v1/solicitudes/{id}/aceptar`	TRABAJADOR, ADMIN	OFY-26	200 OK
PATCH	`/api/v1/solicitudes/{id}/rechazar`	TRABAJADOR, ADMIN	OFY-27	200 OK
PATCH	`/api/v1/solicitudes/{id}/iniciar`	TRABAJADOR, ADMIN	OFY-78	200 OK

Método	Ruta	Rol requerido	RF / HU	Éxito
PATCH	`/api/v1/solicitudes/{id}/completar`	TRABAJADOR, ADMIN	OFY-29 · OFY-78	200 OK
GET	`/api/v1/solicitudes/{id}`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-78	200 OK
PATCH	`/api/v1/solicitudes/{id}/cancelar?motivo=`	CONTRATANTE, ADMIN	OFY-76	200 OK
POST	`/api/v1/solicitudes/{solicitudId}/mensajes`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-51	200 OK
GET	`/api/v1/solicitudes/{solicitudId}/mensajes`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-51	200 OK
PATCH	`/api/v1/solicitudes/{solicitudId}/mensajes/leidos?destinatarioId=`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-51	204 No Content
GET	`/api/v1/historial-estadisticas/cliente/{clienteId}/historial`	CLIENTE (ver L4)	OFY-44 · OFY-47 · OFY-72	200 OK
GET	`/api/v1/historial-estadisticas/trabajador/{trabajadorId}/historial`	TRABAJADOR	OFY-44	200 OK
GET	`/api/v1/historial-estadisticas/cliente/{clienteId}/estadisticas`	CLIENTE (ver L4)	OFY-44	200 OK
GET	`/api/v1/historial-estadisticas/trabajador/{trabajadorId}/estadisticas`	TRABAJADOR	OFY-08 · OFY-44	200 OK
GET	`/api/v1/historial-estadisticas/cliente/{clienteId}/dashboard`	CLIENTE (ver L4)	OFY-44 · OFY-72	200 OK
GET	`/api/v1/historial-estadisticas/trabajador/{trabajadorId}/dashboard`	TRABAJADOR	OFY-08 · OFY-44	200 OK
POST	`/api/v1/reputacion/solicitud/{id}/trabajador?contratanteId=`	CONTRATANTE, ADMIN	OFY-34 · OFY-36 · OFY-37	201 Created
POST	`/api/v1/reputacion/solicitud/{id}/contratante?trabajadorId=`	TRABAJADOR, ADMIN	OFY-35 · OFY-36	201 Created
POST	`/api/v1/reputacion/resena/{resenaId}/reporte`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-38	204 No Content
GET	`/api/v1/reputacion/trabajador/{trabajadorId}/oficio/{oficioId}`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-56 · OFY-71	200 OK
GET	`/api/v1/reputacion/usuario/{usuarioId}`	CONTRATANTE, TRABAJADOR, ADMIN	OFY-10 · OFY-36	200 OK
POST	`/api/v1/moderacion/reportes`	TRABAJADOR, CONTRATANTE	OFY-64	201 Created
GET	`/api/v1/moderacion/reportes/{id}`	TRABAJADOR, CONTRATANTE, ADMIN	OFY-73	200 OK
GET	`/api/v1/moderacion/reportes`	ADMIN	OFY-54	200 OK
PATCH	`/api/v1/moderacion/usuarios/{id}/pausar`	ADMIN	OFY-54	200 OK
POST	`/api/v1/referidos/invitar?correo=`	TRABAJADOR, CONTRATANTE	OFY-68	200 OK

5.3 Autenticación `/auth`

Es la puerta de entrada. Recibe las credenciales, las verifica contra PostgreSQL con BCrypt y entrega un JWT firmado con HS256 que dura 24 horas. El token lleva el correo en `sub` y el rol en el claim `role`.

POST `/auth/login`

Rol: Público **Requerimiento:** OFY-50 **Respuesta exitosa:** 200 OK

Inicia sesión y devuelve el token de acceso.

- **Cuerpo:** AuthRequest
- **Respuesta:** AuthResponse
- **Errores:** 401 credenciales inválidas o cuenta pausada

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiUk9MRV9DT05UUkFUQU5URSIs..."
}
```

POST `/auth/switch-role`

Rol: Cualquier token válido (cabecera Authorization) **Requerimiento:** OFY-55 **Respuesta exitosa:** 200 OK

Genera un token nuevo con el rol que el usuario quiere usar (modo Contratante o modo Profesional).

- **Cuerpo:** SwitchRoleRequest
- **Respuesta:** AuthResponse
- **Errores:** Sin token o con token inválido no responde un 401 limpio (ver L2)

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiUk9MRV9DT05UUkFUQU5URSIs..."
}
```

⚠ Hoy acepta cualquier rol, incluido ADMIN. Ver limitación L3.

5.4 Salud del servicio `/`

Endpoints de monitoreo que no necesitan token. Sirven para comprobar en segundos, el día de la feria, que la API está viva.

`GET` **`/api/v1/health`**

Rol: Público **Requerimiento:** Soporte técnico **Respuesta exitosa:** 200 OK

Devuelve el estado del servicio.

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "status": "UP",
  "service": "OficioYa-Backend",
  "timestamp": "2026-10-20T13:00:00Z"
}
```

`GET` **`/`**

Rol: Público **Requerimiento:** Soporte técnico **Respuesta exitosa:** 302 Found

Redirige a la interfaz de Swagger (`/swagger-ui/index.html`).

RESPUESTA DE EJEMPLO · 302 FOUND

Sin cuerpo. La respuesta trae la cabecera `Location: /swagger-ui/index.html`.

5.5 Usuarios e identidad `/api/v1/usuarios`

Maneja la cuenta de la persona: registro, verificación de correo, consulta y edición de sus perfiles, foto, baja lógica y recuperación.

`POST` **`/api/v1/usuarios/registro`**

Rol: Público (sin token) **Requerimiento:** OFY-49 **Respuesta exitosa:** 201 Created

Crea la cuenta. Si el rol es CONTRATANTE crea también su perfil de contratante y, si el correo había sido invitado, acredita el referido.

- **Cuerpo:** UsuarioRegistroRequestDTO
- **Respuesta:** UsuarioResponseDTO
- **Errores:** 400 validación · 409 correo ya registrado

RESPUESTA DE EJEMPLO · 201 CREATED

```
{
  "id": 4,
  "correo": "ana.ruiz@correo.com",
  "telefono": "3001112233",
  "nombre": "Ana",
  "apellido": "Ruiz",
  "rol": "CONTRATANTE",
  "correoVerificado": false,
  "telefonoVerificado": false,
  "fechaRegistro": "2026-10-20T13:05:10",
  "activo": true
}
```

`PATCH` **`/api/v1/usuarios/{correo}/verificar`**

Rol: TRABAJADOR, CONTRATANTE, ADMIN **Requerimiento:** OFY-40 **Respuesta exitosa:** 200 OK

Marca el correo como verificado.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404 correo inexistente · 422 ya estaba verificado

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

GET /api/v1/usuarios/{id}/perfil-trabajador

Rol: TRABAJADOR, CONTRATANTE, ADMIN **Requerimiento:** OFY-52 **Respuesta exitosa:** 200 OK

Consulta el perfil profesional asociado a un usuario.

- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 404 usuario o perfil inexistente · 422 el usuario no es trabajador

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

GET /api/v1/usuarios/{id}/perfil-contratante

Rol: TRABAJADOR, CONTRATANTE, ADMIN **Requerimiento:** OFY-53 **Respuesta exitosa:** 200 OK

Consulta el perfil de contratante de un usuario.

- **Respuesta:** PerfilContratanteResponseDTO
- **Errores:** 404 · 422 el usuario no es contratante

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "id": 3,
  "usuarioId": 4,
  "calificacionPromedio": 4.5,
  "serviciosSolicitados": 6,
  "descripcion": "Vivo en Cedritos y suelo pedir arreglos los fines de semana."
}
```

PUT /api/v1/usuarios/{id}/perfil-trabajador

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-57 **Respuesta exitosa:** 200 OK

Edita teléfono y zona de cobertura del trabajador.

- **Cuerpo:** EditarPerfilTrabajadorRequestDTO
- **Respuesta:** MensajeResponseDTO
- **Errores:** 400 · 404 · 422 el usuario no es trabajador

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

PUT /api/v1/usuarios/{id}/perfil-contratante

Rol: CONTRATANTE, ADMIN **Requerimiento:** OFY-57 **Respuesta exitosa:** 200 OK

Edita teléfono y descripción del contratante.

- **Cuerpo:** EditarPerfilContratanteRequestDTO
- **Respuesta:** MensajeResponseDTO
- **Errores:** 400 · 404 · 422 el usuario no es contratante

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

DELETE `/api/v1/usuarios/{id}`

Rol: TRABAJADOR, CONTRATANTE **Requerimiento:** OFY-58 **Respuesta exitosa:** 200 OK

Baja lógica: la cuenta queda con `activo = false` y se conserva su historial.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404 · 422 la cuenta ya estaba dada de baja

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

POST `/api/v1/usuarios/{id}/foto`

Rol: TRABAJADOR, CONTRATANTE **Requerimiento:** OFY-59 **Respuesta exitosa:** 200 OK

Sube la foto de perfil. Formulario `multipart/form-data` con la parte `file` (JPG o PNG, máximo 5 MB).

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404 · 422 archivo vacío, formato no permitido o cuenta inactiva

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

POST `/api/v1/usuarios/recuperar-cuenta?correo=`

Rol: Público (sin token) **Requerimiento:** OFY-75 **Respuesta exitosa:** 200 OK

Inicia la recuperación de la cuenta. Responde con un mensaje neutro para no revelar si el correo existe.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404 · 422 cuenta inactiva

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

5.6 Perfil del trabajador `/api/v1/perfiles-trabajador`

Es el corazón de la oferta: aquí el trabajador arma su perfil profesional paso a paso (oficios, zona, tarifa, horario, pagos, portafolio). Todas las operaciones de escritura exigen el rol TRABAJADOR.

POST `/api/v1/perfiles-trabajador`

Rol: TRABAJADOR **Requerimiento:** OFY-01 **Respuesta exitosa:** 201 Created

Crea el perfil profesional del usuario.

- **Cuerpo:** PerfilTrabajadorCreacionRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404 usuario inexistente · 409 ya tiene perfil · 422 no es trabajador, está inactivo o sin verificar

RESPUESTA DE EJEMPLO · 201 CREATED

```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

PUT `/api/v1/perfiles-trabajador/{id}/oficio-principal`

Rol: TRABAJADOR **Requerimiento:** OFY-02 **Respuesta exitosa:** 200 OK

Asigna el oficio principal.

- **Cuerpo:** OficioPrincipalRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404 perfil u oficio inexistente · 422 oficio inactivo

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PUT `/api/v1/perfiles-trabajador/{id}/oficios-secundarios`

Rol: TRABAJADOR **Requerimiento:** OFY-03 **Respuesta exitosa:** 200 OK

Reemplaza la lista de oficios secundarios (máximo 5, sin repetir el principal).

- **Cuerpo:** OficiosSecundariosRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404 · 422 oficio repetido o inactivo

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PUT `/api/v1/perfiles-trabajador/{id}/zona-cobertura`

Rol: TRABAJADOR **Requerimiento:** OFY-04 **Respuesta exitosa:** 200 OK

Define los barrios o localidades que atiende.

- **Cuerpo:** ZonaCoberturaRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

PUT /api/v1/perfiles-trabajador/{id}/tarifa

Rol: TRABAJADOR **Requerimiento:** OFY-05 **Respuesta exitosa:** 200 OK

Registra la tarifa aproximada por hora en pesos colombianos.

- **Cuerpo:** TarifaRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PUT /api/v1/perfiles-trabajador/{id}/disponibilidad-semanal

Rol: TRABAJADOR **Requerimiento:** OFY-06 **Respuesta exitosa:** 200 OK

Reemplaza las franjas semanales de atención.

- **Cuerpo:** DisponibilidadSemanalRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404 · 422 hora de inicio no anterior a la de fin o franjas solapadas

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PATCH /api/v1/perfiles-trabajador/{id}/portafolio

Rol: TRABAJADOR **Requerimiento:** OFY-07 **Respuesta exitosa:** 200 OK

Guarda las URLs de las fotos del portafolio en MongoDB.

- **Cuerpo:** PortafolioRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 más de 20 fotos · 404 · 500 si MongoDB no responde

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

PATCH `/api/v1/perfiles-trabajador/{id}/detalles-especificos`

Rol: TRABAJADOR **Requerimiento:** OFY-09 **Respuesta exitosa:** 200 OK

Texto libre con los detalles del oficio (marcas, materias, certificaciones).

- **Cuerpo:** DetallesEspecificosRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Criticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PATCH `/api/v1/perfiles-trabajador/{id}/disponible-ahora/activar`

Rol: TRABAJADOR **Requerimiento:** OFY-30 **Respuesta exitosa:** 200 OK

Enciende el interruptor "Disponible ahora" para recibir urgencias.

- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 404 · 422 ya estaba activo o no tiene zona de cobertura

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Criticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


PATCH `/api/v1/perfiles-trabajador/{id}/disponible-ahora/desactivar`

Rol: TRABAJADOR **Requerimiento:** OFY-31 **Respuesta exitosa:** 200 OK

Apaga el interruptor "Disponible ahora".

- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 404 · 422 no estaba activo

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Criticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

PUT `/api/v1/perfiles-trabajador/{id}/metodos-pago`

Rol: TRABAJADOR **Requerimiento:** OFY-60 **Respuesta exitosa:** 200 OK

Reemplaza los métodos de pago aceptados.

- **Cuerpo:** MetodosPagoRequestDTO
- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 400 · 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


GET `/api/v1/perfiles-trabajador/{id}`

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-65 **Respuesta exitosa:** 200 OK

Perfil público completo, con el portafolio sincronizado desde MongoDB.

- **Respuesta:** PerfilTrabajadorResponseDTO
- **Errores:** 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```


GET `/api/v1/perfiles-trabajador/{id}/oficios/{oficioId}/especializaciones`

Rol: TRABAJADOR, CONTRATANTE **Requerimiento:** OFY-66 **Respuesta exitosa:** 200 OK

Lista las especializaciones registradas para un oficio.

- **Errores:** 404

RESPUESTA DE EJEMPLO · 200 OK


```
[
  "Grifería",
  "Calentadores a gas"
]
```


PUT `/api/v1/perfiles-trabajador/{id}/oficios/{oficioId}/especializaciones`

Rol: TRABAJADOR **Requerimiento:** OFY-67 **Respuesta exitosa:** 200 OK

Reemplaza las especializaciones de un oficio. El cuerpo es un arreglo JSON de textos.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404

CUERPO DE EJEMPLO


```
["Grifería", "Calentadores a gas"]
```


RESPUESTA DE EJEMPLO · 200 OK


```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

5.7 Verificación de identidad `/api/v1/trabajadores/verificacion`

Recibe los documentos con los que un trabajador pide la insignia de identidad verificada. Cuando el perfil tiene los tres documentos, el estado pasa solo de NO_VERIFICADO (o RECHAZADO) a PENDIENTE_REVISION.

POST `/api/v1/trabajadores/verificacion/{trabajadorId}/documentos`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-42 **Respuesta exitosa:** 200 OK

Multipart con las partes `frente` y `reverso` del documento. Devuelve las dos URLs separadas por punto y coma.

- **Errores:** 404 perfil inexistente · 422 archivo vacío o formato no permitido

RESPUESTA DE EJEMPLO · 200 OK

```
"https://mock.cloudinary.com/oficioya/image/upload/v1/9f1c...jpg;https://mock.cloudinary.com/oficioya/image/upload/v1/3a7e...jpg"
```

POST `/api/v1/trabajadores/verificacion/{trabajadorId}/antecedentes`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-42 **Respuesta exitosa:** 200 OK

Multipart con la parte `certificado`. Devuelve la URL guardada.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

```
"https://mock.cloudinary.com/oficioya/image/upload/v1/antecedentes-2.pdf"
```

POST `/api/v1/trabajadores/verificacion/{trabajadorId}/foto-verificada`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-42 **Respuesta exitosa:** 200 OK

Multipart con la parte `foto`. Devuelve la URL guardada.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

```
"https://mock.cloudinary.com/oficioya/image/upload/v1/selfie-2.jpg"
```

GET `/api/v1/trabajadores/verificacion/{trabajadorId}/estado`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-42 **Respuesta exitosa:** 200 OK

Devuelve el estado actual como texto del enum EstadoVerificacion.

- **Errores:** 404

RESPUESTA DE EJEMPLO · 200 OK

```
"PENDIENTE_REVISION"
```

5.8 Catálogo y búsqueda `/api/v1`

Es lo que ve primero un contratante: el catálogo de oficios y el buscador. El buscador entiende lenguaje natural sencillo ("alguien que arregle una gotera" se traduce en plomería) y combina todos los filtros en una sola consulta SQL. El orden nunca depende de pagos: solo de reputación o de cercanía.

GET `/api/v1/oficios`

Rol: Público (sin token) **Requerimiento:** OFY-77 **Respuesta exitosa:** 200 OK

Lista los oficios activos del catálogo (18 oficios semilla en 6 categorías).

- **Respuesta:** OficioResponseDTO[]

RESPUESTA DE EJEMPLO · 200 OK

```
[
  {
    "id": 1,
    "nombre": "Plomería",
    "categoria": "Emergencias y Reparaciones Críticas",
    "descripcion": "Atención de fugas, destape de cañerías, instalación de grifería y sanitarios.",
    "activo": true
  }
]
```

`GET` **`/api/v1/busqueda/trabajadores`**

Rol: Público (sin token) **Requerimiento:** OFY-11 a OFY-18, OFY-32 **Respuesta exitosa:** 200 OK

Busca trabajadores con filtros opcionales en la query string. La lista puede venir vacía.

- **Parámetros de consulta:** FiltroBusquedaDTO
- **Respuesta:** PerfilTrabajadorResponseDTO[]
- **Errores:** 400 formato inválido · 422 tarifa mínima mayor que la máxima, hora sin día, horaHasta sin horaDesde o inicio no anterior al fin

RESPUESTA DE EJEMPLO · 200 OK


```
[
  {
    "id": 1,
    "usuarioId": 2,
    "descripcion": "Plomero con 10 años de experiencia",
    "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
    "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
    "zonaCobertura": "Chapinero, Usaquén",
    "tarifaPorHora": 35000.00,
    "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
    "metodosPago": ["EFECTIVO", "NEQUI"],
    "calificacionPromedio": 4.7,
    "trabajosCompletados": 12,
    "disponibleAhora": true,
    "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
    "estadoVerificacion": "APROBADO"
  }
]
```


5.9 Solicitudes de servicio `/api/v1/solicitudes`

Gobierna el ciclo de vida de un servicio con el patrón State. Cada endpoint de transición solo funciona desde los estados que lo permiten; si no, la API responde 422 y dice desde qué estado se intentó.

`POST` **`/api/v1/solicitudes`**

Rol: CONTRATANTE, ADMIN **Requerimiento:** OFY-19 · OFY-20 · OFY-21 **Respuesta exitosa:** 201 Created

Crea la solicitud en estado CREADA.

- **Cuerpo:** SolicitudCreacionDTO
- **Respuesta:** SolicitudResponseDTO
- **Errores:** 400 · 404 contratante inexistente · 422 no es contratante o está inactivo

RESPUESTA DE EJEMPLO · 201 CREATED


```
{
  "id": 10,
  "contratanteId": 3,
  "trabajadorId": null,
  "descripcion": "Fuga debajo del lavaplatos",
  "zonaServicio": "Chapinero",
  "fotoAdjuntaUrl": null,
  "estado": "CREADA",
  "fechaCreacion": "2026-10-20T13:10:00"
}
```


`POST` **`/api/v1/solicitudes/{id}/enviar?trabajadorId=`**

Rol: CONTRATANTE, ADMIN **Requerimiento:** OFY-23 · OFY-25 **Respuesta exitosa:** 200 OK

Pasa de CREADA a ENVIADA y publica un SolicitudEvent que dispara la notificación.

- **Errores:** 404 · 422 transición inválida · 500 si falta trabajadorId (ver L8)

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

> ⚠ Hoy responde 500 porque la solicitud leída de la base llega sin objeto de estado (L5). Además el trabajadorId se recibe pero no se guarda (L6).

`PATCH` **`/api/v1/solicitudes/{id}/aceptar`**

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-26 **Respuesta exitosa:** 200 OK

ENVIADA → ACEPTADA.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

> ⚠ Igual que las demás transiciones, hoy responde 500 por L5.

PATCH `/api/v1/solicitudes/{id}/rechazar`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-27 **Respuesta exitosa:** 200 OK

ENVIADA → RECHAZADA.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

PATCH `/api/v1/solicitudes/{id}/iniciar`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-78 **Respuesta exitosa:** 200 OK

ACEPTADA → EN_PROGRESO.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

PATCH `/api/v1/solicitudes/{id}/completar`

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-29 · OFY-78 **Respuesta exitosa:** 200 OK

EN_PROGRESO → COMPLETADA. Desde aquí se habilitan las reseñas.

- **Errores:** 404 · 422

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

GET `/api/v1/solicitudes/{id}`

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-78 **Respuesta exitosa:** 200 OK

Devuelve una solicitud con su estado actual. Sirve para que el front refresque el detalle después de cada transición.

- **Respuesta:** SolicitudResponseDTO
- **Errores:** 404 "Solicitud no existe"

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "id": 10,
  "contratanteId": 3,
  "trabajadorId": null,
  "descripcion": "Fuga debajo del lavaplatos",
  "zonaServicio": "Chapinero",
  "fotoAdjuntaUrl": null,
  "estado": "CREADA",
  "fechaCreacion": "2026-10-20T13:10:00"
}
```


PATCH `/api/v1/solicitudes/{id}/cancelar?motivo=`

Rol: CONTRATANTE, ADMIN **Requerimiento:** OFY-76 **Respuesta exitosa:** 200 OK

ENVIADA o ACEPTADA → CANCELADA. El parámetro `motivo` es obligatorio.

- **Errores:** 404 · 422 · 500 si falta el parámetro (ver L8)

RESPUESTA DE EJEMPLO · 200 OK

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **200 OK**.*

5.10 Mensajería por solicitud `/api/v1/solicitudes/{solicitudId}/mensajes`

Un chat sencillo atado a cada solicitud. Los mensajes viven en la colección `mensajes` de MongoDB y se consultan por REST (sin WebSocket).

POST `/api/v1/solicitudes/{solicitudId}/mensajes`

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-51 **Respuesta exitosa:** 200 OK

Envía un mensaje.

- **Cuerpo:** MensajeChatRequestDTO
- **Respuesta:** MensajeChatResponseDTO
- **Errores:** 400

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "id": "6714f0a2c9e77b1d2a3f9c10",
  "solicitudId": 10,
  "remitenteId": 4,
  "destinatarioId": 2,
  "contenido": "¿Puede venir hoy a las 3 p. m.?",
  "fechaEnvio": "2026-10-20T14:02:00",
  "leido": false
}
```

GET `/api/v1/solicitudes/{solicitudId}/mensajes`

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-51 **Respuesta exitosa:** 200 OK

Lista los mensajes de la solicitud.

- **Respuesta:** MensajeChatResponseDTO[]

RESPUESTA DE EJEMPLO · 200 OK

```
[
  {
    "id": "6714f0a2c9e77b1d2a3f9c10",
    "solicitudId": 10,
    "remitenteId": 4,
    "destinatarioId": 2,
    "contenido": "¿Puede venir hoy a las 3 p. m.?",
    "fechaEnvio": "2026-10-20T14:02:00",
    "leido": false
  }
]
```

PATCH `/api/v1/solicitudes/{solicitudId}/mensajes/leidos?destinatarioId=`

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-51 **Respuesta exitosa:** 204 No Content

Marca como leídos los mensajes dirigidos a ese destinatario.

RESPUESTA DE EJEMPLO · 204 NO CONTENT

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **204 No Content**.*

5.11 Historial y estadísticas `/api/v1/historial-estadisticas`

Le da a cada persona su tablero: lo que ha pedido o atendido y cuántos servicios están completados, cancelados, en progreso o pendientes.

GET `/api/v1/historial-estadisticas/cliente/{clienteId}/historial`

Rol: CLIENTE (ver L4) **Requerimiento:** OFY-44 · OFY-47 · OFY-72 **Respuesta exitosa:** 200 OK

Solicitudes del contratante, de la más reciente a la más antigua.

- **Respuesta:** SolicitudResponseDTO[]

RESPUESTA DE EJEMPLO · 200 OK

```
[
  {
    "id": 10,
    "contratanteId": 3,
    "trabajadorId": null,
    "descripcion": "Fuga debajo del lavaplatos",
    "zonaServicio": "Chapinero",
    "fotoAdjuntaUrl": null,
    "estado": "CREADA",
    "fechaCreacion": "2026-10-20T13:10:00"
  }
]
```

GET `/api/v1/historial-estadisticas/trabajador/{trabajadorId}/historial`

Rol: TRABAJADOR **Requerimiento:** OFY-44 **Respuesta exitosa:** 200 OK

Trabajos del trabajador, del más reciente al más antiguo.

- **Respuesta:** SolicitudResponseDTO[]

RESPUESTA DE EJEMPLO · 200 OK

```
[
  {
    "id": 10,
    "contratanteId": 3,
    "trabajadorId": null,
    "descripcion": "Fuga debajo del lavaplatos",
    "zonaServicio": "Chapinero",
    "fotoAdjuntaUrl": null,
    "estado": "CREADA",
    "fechaCreacion": "2026-10-20T13:10:00"
  }
]
```

GET `/api/v1/historial-estadisticas/cliente/{clienteId}/estadisticas`

Rol: CLIENTE (ver L4) **Requerimiento:** OFY-44 **Respuesta exitosa:** 200 OK

Conteos por estado del contratante.

- **Respuesta:** EstadisticasResponseDTO

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "total": 8,
  "completadas": 5,
  "canceladas": 1,
  "enProgreso": 1,
  "pendientes": 1
}
```

GET `/api/v1/historial-estadisticas/trabajador/{trabajadorId}/estadisticas`

Rol: TRABAJADOR **Requerimiento:** OFY-08 · OFY-44 **Respuesta exitosa:** 200 OK

Conteos por estado del trabajador.

- **Respuesta:** EstadisticasResponseDTO

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "total": 8,
  "completadas": 5,
  "canceladas": 1,
  "enProgreso": 1,
  "pendientes": 1
}
```

GET `/api/v1/historial-estadisticas/cliente/{clienteId}/dashboard`

Rol: CLIENTE (ver L4) **Requerimiento:** OFY-44 · OFY-72 **Respuesta exitosa:** 200 OK

Estadísticas más historial reciente en una sola respuesta.

- **Respuesta:** DashboardResumenDTO

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "estadisticas": { "total": 8, "completadas": 5, "canceladas": 1, "enProgreso": 1, "pendientes": 1 },
  "historialReciente": [
    { "id": 10, "contratanteId": 4, "trabajadorId": 2, "descripcion": "Fuga debajo del lavaplatos",
      "zonaServicio": "Chapinero", "fotoAdjuntaUrl": null, "estado": "COMPLETADA", "fechaCreacion": "2026-10-20T13:10:00" }
  ]
}
```

`GET` **`/api/v1/historial-estadisticas/trabajador/{trabajadorId}/dashboard`**

Rol: TRABAJADOR **Requerimiento:** OFY-08 · OFY-44 **Respuesta exitosa:** 200 OK

Estadísticas más historial reciente en una sola respuesta.

- **Respuesta:** DashboardResumenDTO

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "estadisticas": { "total": 8, "completadas": 5, "canceladas": 1, "enProgreso": 1, "pendientes": 1 },
  "historialReciente": [
    { "id": 10, "contratanteId": 4, "trabajadorId": 2, "descripcion": "Fuga debajo del lavaplatos",
      "zonaServicio": "Chapinero", "fotoAdjuntaUrl": null, "estado": "COMPLETADA", "fechaCreacion": "2026-10-20T13:10:00" }
  ]
}
```


5.12 Reputación y reseñas `/api/v1/reputacion`

La calificación es mutua y solo es posible sobre una solicitud COMPLETADA, así que solo califica quien de verdad recibió o prestó el servicio. El promedio se recalcula con una estrategia intercambiable y descarta las reseñas reportadas.

`POST` **`/api/v1/reputacion/solicitud/{id}/trabajador?contratanteId=`**

Rol: CONTRATANTE, ADMIN **Requerimiento:** OFY-34 · OFY-36 · OFY-37 **Respuesta exitosa:** 201 Created

El contratante califica al trabajador.

- **Cuerpo:** ResenaRequestDTO
- **Respuesta:** ResenaResponseDTO
- **Errores:** 400 · 404 solicitud inexistente · 422 la solicitud no está COMPLETADA o el contratante no es el de la solicitud

RESPUESTA DE EJEMPLO · 201 CREATED


```
{
  "id": 21,
  "autorId": 3,
  "autorNombre": "Laura Cliente",
  "receptorId": 2,
  "receptorNombre": "Carlos Plomero",
  "oficioId": 1,
  "oficioNombre": "Plomería",
  "estrellas": 5,
  "comentario": "Llegó puntual y dejó todo limpio.",
  "fechaCreacion": "2026-10-20T15:40:00"
}
```


`POST` **`/api/v1/reputacion/solicitud/{id}/contratante?trabajadorId=`**

Rol: TRABAJADOR, ADMIN **Requerimiento:** OFY-35 · OFY-36 **Respuesta exitosa:** 201 Created

El trabajador califica al contratante.

- **Cuerpo:** ResenaRequestDTO
- **Respuesta:** ResenaResponseDTO
- **Errores:** 400 · 404 · 422

RESPUESTA DE EJEMPLO · 201 CREATED


```
{
  "id": 21,
  "autorId": 3,
  "autorNombre": "Laura Cliente",
  "receptorId": 2,
  "receptorNombre": "Carlos Plomero",
  "oficioId": 1,
  "oficioNombre": "Plomería",
  "estrellas": 5,
  "comentario": "Llegó puntual y dejó todo limpio.",
  "fechaCreacion": "2026-10-20T15:40:00"
}
```


`POST` **`/api/v1/reputacion/resena/{resenaId}/reporte`**

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-38 **Respuesta exitosa:** 204 No Content

Marca una reseña como reportada y la excluye del promedio.

- **Cuerpo:** ReporteResenaRequestDTO
- **Errores:** 400 · 404 · 422 ya había sido reportada

RESPUESTA DE EJEMPLO · 204 NO CONTENT

*Sin cuerpo de respuesta: el éxito se comunica solo con el código **204 No Content**.*

`GET` **/api/v1/reputacion/trabajador/{trabajadorId}/oficio/{oficioId}**

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-56 · OFY-71 **Respuesta exitosa:** 200 OK

Reputación del trabajador en un oficio concreto.

- **Respuesta:** ReputacionPorOficioDTO

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "trabajadorId": 2,
  "oficioId": 1,
  "oficioNombre": "Oficio 1",
  "calificacionPromedio": 4.7,
  "totalResenas": 12
}
```

`GET` **/api/v1/reputacion/usuario/{usuarioId}**

Rol: CONTRATANTE, TRABAJADOR, ADMIN **Requerimiento:** OFY-10 · OFY-36 **Respuesta exitosa:** 200 OK

Reseñas recibidas por un usuario.

- **Respuesta:** ResenaResponseDTO[]

RESPUESTA DE EJEMPLO · 200 OK

```
[
  {
    "id": 21,
    "autorId": 3,
    "autorNombre": "Laura Cliente",
    "receptorId": 2,
    "receptorNombre": "Carlos Plomero",
    "oficioId": 1,
    "oficioNombre": "Plomería",
    "estrellas": 5,
    "comentario": "Llegó puntual y dejó todo limpio.",
    "fechaCreacion": "2026-10-20T15:40:00"
  }
]
```

5.13 Moderación `/api/v1/moderacion`

Cuida la comunidad. Cualquier usuario puede reportar a otro y el administrador revisa los reportes y puede pausar cuentas. El reportador se toma del token, no del cuerpo.

`POST` **/api/v1/moderacion/reportes**

Rol: TRABAJADOR, CONTRATANTE **Requerimiento:** OFY-64 **Respuesta exitosa:** 201 Created

Radica un reporte en estado PENDIENTE.

- **Cuerpo:** ReporteRequestDTO
- **Respuesta:** ReporteResponseDTO
- **Errores:** 400 · 404 reportador o reportado inexistente

RESPUESTA DE EJEMPLO · 201 CREATED

```
{
  "id": 7,
  "reportadorId": 4,
  "reportadoId": 9,
  "motivo": "Pidió el pago por fuera de la plataforma y no llegó.",
  "evidenciaUrl": "https://mock.cloudinary.com/oficioya/image/upload/v1/chat.png",
  "estado": "PENDIENTE",
  "fechaCreacion": "2026-10-20T16:30:00"
}
```

`GET` **/api/v1/moderacion/reportes/{id}**

Rol: TRABAJADOR, CONTRATANTE, ADMIN **Requerimiento:** OFY-73 **Respuesta exitosa:** 200 OK

Consulta el estado de un reporte.

- **Respuesta:** ReporteResponseDTO
- **Errores:** 404

RESPUESTA DE EJEMPLO · 200 OK

```
{
  "id": 7,
  "reportadorId": 4,
  "reportadoId": 9,
  "motivo": "Pidió el pago por fuera de la plataforma y no llegó.",
  "evidenciaUrl": "https://mock.cloudinary.com/oficioya/image/upload/v1/chat.png",
  "estado": "PENDIENTE",
  "fechaCreacion": "2026-10-20T16:30:00"
}
```

`GET` **`/api/v1/moderacion/reportes`**

Rol: ADMIN **Requerimiento:** OFY-54 **Respuesta exitosa:** 200 OK

Panel de administración: todos los reportes.

- **Respuesta:** ReporteResponseDTO[]
- **Errores:** 403 si el rol no es ADMIN

RESPUESTA DE EJEMPLO · 200 OK


```
[
  {
    "id": 7,
    "reportadorId": 4,
    "reportadoId": 9,
    "motivo": "Pidió el pago por fuera de la plataforma y no llegó.",
    "evidenciaUrl": "https://mock.cloudinary.com/oficioya/image/upload/v1/chat.png",
    "estado": "PENDIENTE",
    "fechaCreacion": "2026-10-20T16:30:00"
  }
]
```


`PATCH` **`/api/v1/moderacion/usuarios/{id}/pausar`**

Rol: ADMIN **Requerimiento:** OFY-54 **Respuesta exitosa:** 200 OK

Pausa la cuenta (`activo = false`); el usuario ya no puede iniciar sesión.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 403 · 404

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```


5.14 Referidos `/api/v1/referidos`

Premia a quien trae comunidad. Cuando cinco invitados de una persona crean su cuenta, esa persona recibe la insignia de embajador.

`POST` **`/api/v1/referidos/invitar?correo=`**

Rol: TRABAJADOR, CONTRATANTE **Requerimiento:** OFY-68 **Respuesta exitosa:** 200 OK

Registra la invitación a un correo. El referente se toma del token.

- **Respuesta:** MensajeResponseDTO
- **Errores:** 404 · 422 el correo ya tiene cuenta o ya fue invitado

RESPUESTA DE EJEMPLO · 200 OK


```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```


5.15 Modelos de request y response

Cada endpoint de arriba enlaza a su modelo. Aquí está cada campo con su tipo, si es obligatorio y sus restricciones, tomadas de las anotaciones de validación del código. Los objetos anidados tienen su propia tabla.

Enumeraciones

Enum	Valores	Dónde se usa
RolUsuario	CONTRATANTE · TRABAJADOR · ADMIN	Registro, token, autorización
EstadoSolicitud	CREADA · ENVIADA · ACEPTADA · RECHAZADA · EN_PROGRESO · COMPLETADA · CANCELADA	Solicitudes, historial
EstadoVerificacion	NO_VERIFICADO · PENDIENTE_REVISION · APROBADO · RECHAZADO	Perfil y verificación
MetodoPago	EFECTIVO · NEQUI · DAVIPLATA	Perfil del trabajador
EstadoReporte	PENDIENTE · EN_REVISION · RESUELTO · DESESTIMADO	Moderación
DayOfWeek	MONDAY · TUESDAY · WEDNESDAY · THURSDAY · FRIDAY · SATURDAY · SUNDAY	Disponibilidad y búsqueda
Orden de búsqueda	REPUTACION · DISTANCIA (sin distinguir mayúsculas)	Búsqueda

`AuthRequest`

Credenciales de inicio de sesión.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`email`	String	Sí	Correo registrado	Usuario de la cuenta

Campo	Tipo	Obligatorio	Restricciones	Descripción
`password`	String	Sí	—	Contraseña en texto plano; se compara con el hash BCrypt

EJEMPLO

```
{
  "email": "contratante@oficioya.com",
  "password": "password123"
}
```

AuthResponse

Token de acceso.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`token`	String (JWT)	Sí	HS256 · expira en 24 h	Se envía como `Authorization: Bearer <token>`

EJEMPLO

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiUk9MRV9DT05UUkFUQU5URSIs..."
}
```

SwitchRoleRequest

Rol al que se quiere cambiar.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`newRole`	String	Sí	TRABAJADOR · CONTRATANTE (con o sin prefijo ROLE_)	Rol del nuevo token

EJEMPLO

```
{
  "newRole": "TRABAJADOR"
}
```

UsuarioRegistroRequestDTO

Datos para crear una cuenta.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`correo`	String	Sí	Formato de correo válido · único	Correo de acceso
`telefono`	String	Sí	10 a 15 caracteres · único	Celular de contacto
`contrasena`	String	Sí	Mínimo 6 caracteres	Se guarda cifrada con BCrypt
`nombre`	String	Sí	No vacío	Nombre
`apellido`	String	Sí	No vacío	Apellido
`rol`	Enum RolUsuario	Sí	CONTRATANTE · TRABAJADOR · ADMIN	Rol inicial de la cuenta

EJEMPLO

```
{
  "correo": "ana.ruiz@correo.com",
  "telefono": "3001112233",
  "contrasena": "secreta1",
  "nombre": "Ana",
  "apellido": "Ruiz",
  "rol": "CONTRATANTE"
}
```

UsuarioResponseDTO

Cuenta creada o consultada (nunca incluye la contraseña).

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	Long	Sí	—	Identificador
`correo`	String	Sí	—	Correo
`telefono`	String	Sí	—	Teléfono
`nombre / apellido`	String	Sí	—	Nombre completo
`rol`	Enum RolUsuario	Sí	—	Rol
`correoVerificado / telefonoVerificado`	boolean	Sí	—	Estado de verificación

Campo	Tipo	Obligatorio	Restricciones	Descripción
`fechaRegistro`	LocalDateTime	Sí	ISO-8601	Fecha de alta
`activo`	boolean	Sí	—	false si la cuenta fue dada de baja o pausada

EJEMPLO

```
{
  "id": 4,
  "correo": "ana.ruiz@correo.com",
  "telefono": "3001112233",
  "nombre": "Ana",
  "apellido": "Ruiz",
  "rol": "CONTRATANTE",
  "correoVerificado": false,
  "telefonoVerificado": false,
  "fechaRegistro": "2026-10-20T13:05:10",
  "activo": true
}
```

EditarPerfilTrabajadorRequestDTO

Edición básica del trabajador.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`telefono`	String	Sí	10 a 15 caracteres	Nuevo teléfono
`zonaCobertura`	String	Sí	Máximo 255 caracteres	Nueva zona

EJEMPLO

```
{
  "telefono": "3104445566",
  "zonaCobertura": "Chapinero, Teusaquillo"
}
```

EditarPerfilContratanteRequestDTO

Edición básica del contratante.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`telefono`	String	Sí	10 a 15 caracteres	Nuevo teléfono
`descripcion`	String	No	Máximo 1000 caracteres	Presentación

EJEMPLO

```
{
  "telefono": "3001112233",
  "descripcion": "Vivo en Cedritos y suelo pedir arreglos los fines de semana."
}
```

MensajeResponseDTO

Confirmación en lenguaje cercano.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`mensaje`	String	Sí	—	Texto para mostrar al usuario

EJEMPLO

```
{
  "mensaje": "Perfil de trabajador actualizado exitosamente"
}
```

PerfilContratanteResponseDTO

Perfil del contratante.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	Long	Sí	—	Identificador del perfil
`usuarioId`	Long	Sí	—	Usuario dueño
`calificacionPromedio`	Double	Sí	0.0 a 5.0	Reputación como contratante
`serviciosSolicitados`	Integer	Sí	≥ 0	Servicios pedidos
`descripcion`	String	No	—	Presentación

EJEMPLO

```
{
  "id": 3,
  "usuarioId": 4,
  "calificacionPromedio": 4.5,
  "serviciosSolicitados": 6,
  "descripcion": "Vivo en Cedritos y suelo pedir arreglos los fines de semana."
}
```

PerfilTrabajadorCreacionRequestDTO

Alta del perfil profesional.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`usuarioId`	Long	Sí	Positivo	Usuario con rol TRABAJADOR, activo y con correo o teléfono verificado
`descripcion`	String	No	Máximo 1000 caracteres	Presentación breve

EJEMPLO

```
{
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia en Chapinero"
}
```

OficioPrincipalRequestDTO

Oficio principal.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`oficioId`	Long	Sí	Positivo · oficio activo del catálogo	Oficio principal

EJEMPLO

```
{
  "oficioId": 1
}
```

OficiosSecundariosRequestDTO

Oficios secundarios.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`oficiosIds`	List<Long>	Sí	Máximo 5 · no puede incluir el principal	Reemplaza la lista anterior

EJEMPLO

```
{
  "oficiosIds": [2, 3]
}
```

ZonaCoberturaRequestDTO

Zona de atención.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`zonaCobertura`	String	Sí	No vacío · máximo 255 caracteres	Barrios o localidades, por ejemplo "Chapinero, Usaquén"

EJEMPLO

```
{
  "zonaCobertura": "Chapinero, Usaquén"
}
```

TarifaRequestDTO

Tarifa por hora.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`tarifaPorHora`	BigDecimal	Sí	≥ 0.01 · hasta 8 enteros y 2 decimales	Valor en COP

EJEMPLO

```
{
  "tarifaPorHora": 35000
}
```

DisponibilidadSemanalRequestDTO

Horario semanal. Cada elemento es un objeto FranjaDisponibilidadRequestDTO.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`franjas`	List<FranjaDisponibilidadRequestDTO>	Sí	Máximo 28 franjas · sin solapes en el mismo día	Reemplaza el horario anterior

EJEMPLO

```
{
  "franjas": [
    { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "12:00" },
    { "dia": "MONDAY", "horaInicio": "14:00", "horaFin": "18:00" }
  ]
}
```

FranjaDisponibilidadRequestDTO

Objeto anidado: una franja de atención.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`dia`	Enum DayOfWeek	Sí	MONDAY ... SUNDAY	Día de la semana
`horaInicio`	LocalTime	Sí	HH:mm · anterior a horaFin	Inicio
`horaFin`	LocalTime	Sí	HH:mm	Fin

EJEMPLO

```
{
  "dia": "SATURDAY",
  "horaInicio": "08:00",
  "horaFin": "12:00"
}
```

MetodosPagoRequestDTO

Medios de pago.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`metodosPago`	Set<Enum MetodoPago>	Sí	Al menos uno · EFECTIVO · NEQUI · DAVIPLATA	Reemplaza los anteriores

EJEMPLO

```
{
  "metodosPago": ["EFECTIVO", "NEQUI"]
}
```

DetallesEspecificosRequestDTO

Detalle libre del oficio.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`detallesEspecificos`	String	Sí	No vacío · máximo 1000 caracteres	Por ejemplo marcas que repara o materias que enseña

EJEMPLO

```
{
  "detallesEspecificos": "Trabajo con tubería PVC y cobre; llevo mis herramientas."
}
```

PortafolioRequestDTO

Fotos del portafolio.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`fotosPortafolio`	List<String>	Sí	Máximo 20 URLs	Imágenes ya alojadas (Cloudinary)

EJEMPLO

```
{
  "fotosPortafolio": [
    "https://mock.cloudinary.com/oficioya/image/upload/v1/bano.jpg",
    "https://mock.cloudinary.com/oficioya/image/upload/v1/cocina.jpg"
  ]
}
```

PerfilTrabajadorResponseDTO

Perfil profesional completo. Incluye objetos anidados OficioResumenResponseDTO y FranjaDisponibilidadResponseDTO.

Campo	Tipo	Obligatorio	Restricciones	Descripción
id	Long	Sí	—	Identificador del perfil
usuarioId	Long	Sí	—	Usuario dueño
descripcion	String	No	—	Presentación
oficioPrincipal	OficioResumenResponseDTO	No	Objeto anidado	Oficio principal
oficios	List<OficioResumenResponseDTO>	No	Objeto anidado	Oficios secundarios
zonaCobertura	String	No	—	Zona de atención
tarifaPorHora	BigDecimal	No	COP	Tarifa aproximada
disponibilidadSemanal	List<FranjaDisponibilidadResponseDTO>	No	Objeto anidado	Horario
metodosPago	Set<Enum MetodoPago>	No	—	Medios aceptados
calificacionPromedio	Double	Sí	0.0 a 5.0 · 1 decimal	Reputación
trabajosCompletados	Integer	Sí	≥ 0	Servicios terminados
disponibleAhora	Boolean	Sí	—	Interruptor de urgencias
fotosPortafolio	List<String>	No	Desde MongoDB	Portafolio
documentosIdentidadUrl, certificadoAntecedentesUrl, fotoPerfilVerificadaUrl	String	No	—	Soportes de verificación
estadoVerificacion	Enum EstadoVerificacion	Sí	—	Estado de la insignia

EJEMPLO

```
{
  "id": 1,
  "usuarioId": 2,
  "descripcion": "Plomero con 10 años de experiencia",
  "oficioPrincipal": { "id": 1, "nombre": "Plomería", "categoria": "Emergencias y Reparaciones Críticas" },
  "oficios": [ { "id": 3, "nombre": "Cerrajería", "categoria": "Emergencias y Reparaciones Críticas" } ],
  "zonaCobertura": "Chapinero, Usaquén",
  "tarifaPorHora": 35000.00,
  "disponibilidadSemanal": [ { "dia": "MONDAY", "horaInicio": "08:00", "horaFin": "17:00" } ],
  "metodosPago": ["EFECTIVO", "NEQUI"],
  "calificacionPromedio": 4.7,
  "trabajosCompletados": 12,
  "disponibleAhora": true,
  "fotosPortafolio": ["https://res.cloudinary.com/.../bano.jpg"],
  "estadoVerificacion": "APROBADO"
}
```

OficioResumenResponseDTO

Objeto anidado con el resumen de un oficio.

Campo	Tipo	Obligatorio	Restricciones	Descripción
id	Long	Sí	—	Identificador
nombre	String	Sí	—	Nombre del oficio
categoria	String	Sí	—	Categoría

EJEMPLO

```
{
  "id": 1,
  "nombre": "Plomería",
  "categoria": "Emergencias y Reparaciones Críticas"
}
```

FranjaDisponibilidadResponseDTO

Objeto anidado con una franja.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`dia`	Enum DayOfWeek	Sí	MONDAY … SUNDAY	Día
`horaInicio / horaFin`	LocalTime	Sí	HH:mm	Rango horario

EJEMPLO

```
{
  "dia": "MONDAY",
  "horaInicio": "08:00",
  "horaFin": "17:00"
}
```

OficioResponseDTO

Oficio del catálogo.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	Long	Sí	—	Identificador
`nombre`	String	Sí	Único	Por ejemplo Plomería
`categoria`	String	Sí	—	Por ejemplo Emergencias y Reparaciones Críticas
`descripcion`	String	No	—	Qué incluye
`activo`	boolean	Sí	—	Solo se listan los activos

EJEMPLO

```
[
  {
    "id": 1,
    "nombre": "Plomería",
    "categoria": "Emergencias y Reparaciones Críticas",
    "descripcion": "Atención de fugas, destape de cañerías, instalación de grifería y sanitarios.",
    "activo": true
  }
]
```

FiltroBusquedaDTO

Parámetros de consulta (todos opcionales).

Campo	Tipo	Obligatorio	Restricciones	Descripción
`texto`	String	No	Máximo 200 caracteres	OFY-11 · necesidad en lenguaje natural
`categoria`	String	No	—	OFY-12 · categoría del oficio
`oficioId`	Long	No	—	OFY-12 · oficio exacto
`zona`	String	No	—	OFY-13 · barrio o localidad
`tarifaMin / tarifaMax`	BigDecimal	No	≥ 0 · min ≤ max	OFY-14 · rango de tarifa
`dia`	Enum DayOfWeek	No	Obligatorio si se envían horas	OFY-15 · día
`horaDesde / horaHasta`	LocalTime	No	HH:mm · desde < hasta	OFY-15 · franja
`soloDisponiblesAhora`	boolean	No	Por defecto false	OFY-32 · solo urgencias
`calificacionMinima`	Double	No	0.0 a 5.0	OFY-16 · reputación mínima
`orden`	String	No	REPUTACION (por defecto) · DISTANCIA	OFY-17/18 · criterio de orden

EJEMPLO

```
GET /api/v1/busqueda/trabajadores?texto=se%20me%20da%C3%B1%C3%B3%20la%20ducha&zona=Chapinero&tarifaMax=50000&calificacionMinima=4&orden=REPUTACION
```

SolicitudCreacionDTO

Nueva solicitud de servicio.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`contratanteId`	Long	Sí	Usuario CONTRATANTE activo	Quien pide el servicio
`descripcion`	String	Sí	No vacío	Qué necesita

Campo	Tipo	Obligatorio	Restricciones	Descripción
`zonaServicio`	String	Sí	No vacío	Dónde
`fotoAdjuntaUrl`	String	No	URL	Foto del daño

EJEMPLO

```
{
  "contratanteId": 3,
  "descripcion": "Fuga debajo del lavaplatos",
  "zonaServicio": "Chapinero"
}
```

SolicitudResponseDTO

Solicitud.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	Long	Sí	—	Identificador
`contratanteId`	Long	Sí	—	Contratante
`trabajadorId`	Long	No	Nulo mientras nadie la toma	Trabajador
`descripcion, zonaServicio, fotoAdjuntaUrl`	String	—	—	Datos del servicio
`estado`	String (EstadoSolicitud)	Sí	CREADA · ENVIADA · ACEPTADA · RECHAZADA · EN_PROGRESO · COMPLETADA · CANCELADA	Estado actual
`fechaCreacion`	LocalDateTime	Sí	ISO-8601	Alta

EJEMPLO

```
{
  "id": 10,
  "contratanteId": 3,
  "trabajadorId": null,
  "descripcion": "Fuga debajo del lavaplatos",
  "zonaServicio": "Chapinero",
  "fotoAdjuntaUrl": null,
  "estado": "CREADA",
  "fechaCreacion": "2026-10-20T13:10:00"
}
```

MensajeChatRequestDTO

Mensaje de chat.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`remitenteId`	Long	Sí	—	Quien escribe
`destinatarioId`	Long	Sí	—	Quien recibe
`contenido`	String	Sí	No vacío	Texto

EJEMPLO

```
{
  "remitenteId": 3,
  "destinatarioId": 2,
  "contenido": "¿Puede venir hoy a las 4?"
}
```

MensajeChatResponseDTO

Mensaje guardado en MongoDB.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	String (ObjectId)	Sí	—	Identificador del documento
`solicitudId, remitenteId, destinatarioId`	Long	Sí	—	Contexto
`contenido`	String	Sí	—	Texto
`fechaEnvio`	LocalDateTime	Sí	—	Fecha
`leido`	boolean	Sí	—	Lectura

EJEMPLO

```
{
  "id": "6714f0a2c9e77b1d2a3f9c10",
  "solicitudId": 10,
  "remitenteId": 4,
  "destinatarioId": 2,
  "contenido": "¿Puede venir hoy a las 3 p. m.?",
  "fechaEnvio": "2026-10-20T14:02:00",
  "leido": false
}
```

EstadisticasResponseDTO

Conteos por estado.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`total`	long	Sí	—	Todas las solicitudes
`completadas / canceladas / enProgreso`	long	Sí	—	Por estado
`pendientes`	long	Sí	total − (completadas + canceladas + enProgreso)	El resto

EJEMPLO

```
{
  "total": 8,
  "completadas": 5,
  "canceladas": 1,
  "enProgreso": 1,
  "pendientes": 1
}
```

DashboardResumenDTO

Resumen para el tablero.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`estadisticas`	EstadisticasResponseDTO	Sí	Objeto anidado	Conteos
`historialReciente`	List<SolicitudResponseDTO>	Sí	Objeto anidado	Últimas solicitudes

EJEMPLO

```
{
  "estadisticas": { "total": 8, "completadas": 5, "canceladas": 1, "enProgreso": 1, "pendientes": 1 },
  "historialReciente": [
    { "id": 10, "contratanteId": 4, "trabajadorId": 2, "descripcion": "Fuga debajo del lavaplatos",
      "zonaServicio": "Chapinero", "fotoAdjuntaUrl": null, "estado": "COMPLETADA", "fechaCreacion": "2026-10-20T13:10:00" }
  ]
}
```

ResenaRequestDTO

Nueva reseña.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`estrellas`	Integer	Sí	1 a 5	Calificación
`comentario`	String	Sí	No vacío	Opinión

EJEMPLO

```
{
  "estrellas": 5,
  "comentario": "Llegó puntual y dejó todo limpio."
}
```

ResenaResponseDTO

Reseña registrada.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id`	Long	Sí	—	Identificador
`autorId, autorNombre, autorFoto`	Long / String	—	—	Quien califica
`receptorId, receptorNombre`	Long / String	Sí	—	Quien recibe
`oficioId, oficioNombre`	Long / String	Sí	—	Oficio principal del trabajador
`estrellas`	Integer	Sí	1 a 5	Calificación
`comentario`	String	Sí	—	Opinión

Campo	Tipo	Obligatorio	Restricciones	Descripción
`fechaCreacion`	LocalDateTime	Sí	—	Fecha

EJEMPLO

```
{
  "id": 21,
  "autorId": 3,
  "autorNombre": "Laura Cliente",
  "receptorId": 2,
  "receptorNombre": "Carlos Plomero",
  "oficioId": 1,
  "oficioNombre": "Plomería",
  "estrellas": 5,
  "comentario": "Llegó puntual y dejó todo limpio.",
  "fechaCreacion": "2026-10-20T15:40:00"
}
```

ReporteResenaRequestDTO

Motivo para reportar una reseña.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`motivo`	String	Sí	No vacío	Por qué es injusta o inapropiada

EJEMPLO

```
{
  "motivo": "La reseña es de una persona que nunca contrató el servicio."
}
```

ReputacionPorOficioDTO

Reputación en un oficio.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`trabajadorId, oficioId`	Long	Sí	—	Contexto
`oficioNombre`	String	Sí	—	Ver limitación L10
`calificacionPromedio`	Double	Sí	1 decimal	Promedio en ese oficio
`totalResenas`	Long	Sí	—	Reseñas no reportadas

EJEMPLO

```
{
  "trabajadorId": 2,
  "oficioId": 1,
  "oficioNombre": "Oficio 1",
  "calificacionPromedio": 4.7,
  "totalResenas": 12
}
```

ReporteRequestDTO

Reporte de un usuario.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`reportadoId`	Long	Sí	Usuario existente	A quién se reporta
`motivo`	String	Sí	No vacío · máximo 500 caracteres	Qué pasó
`evidenciaUrl`	String	No	URL	Soporte

EJEMPLO

```
{
  "reportadoId": 2,
  "motivo": "No se presentó y pidió pago por adelantado",
  "evidenciaUrl": null
}
```

ReporteResponseDTO

Reporte radicado.

Campo	Tipo	Obligatorio	Restricciones	Descripción
`id, reportadorId, reportadoId`	Long	Sí	—	Identificadores

Campo	Tipo	Obligatorio	Restricciones	Descripción
`motivo, evidenciaUrl`	String	—	—	Contenido
`estado`	String (EstadoReporte)	Sí	PENDIENTE · EN_REVISION · RESUELTO · DESESTIMADO	Estado
`fechaCreacion`	LocalDateTime	Sí	—	Fecha

EJEMPLO


```
{
  "id": 7,
  "reportadorId": 4,
  "reportadoId": 9,
  "motivo": "Pidió el pago por fuera de la plataforma y no llegó.",
  "evidenciaUrl": "https://mock.cloudinary.com/oficioya/image/upload/v1/chat.png",
  "estado": "PENDIENTE",
  "fechaCreacion": "2026-10-20T16:30:00"
}
```

6. Manejo de errores

Todos los errores pasan por `GlobalExceptionHandler` (`@RestControllerAdvice`). Las excepciones de negocio tienen nombre propio y cada una se traduce a un único código HTTP, así el front puede reaccionar sin leer el texto. El mensaje está en español y en tono cercano, porque la SPA lo muestra tal cual (con Many) en sus estados de error.

6.1 Formato de la respuesta: ErrorResponseDTO

Campo	Tipo	Descripción
`status`	int	Código HTTP repetido en el cuerpo
`error`	String	Categoría corta del error
`mensaje`	String	Explicación para la persona
`path`	String	Ruta que se llamó
`timestamp`	LocalDateTime	Momento del error

```
{
  "status": 422,
  "error": "Transición de estado inválida",
  "mensaje": "No se puede ACEPTAR la solicitud desde el estado CREADA",
  "path": "/api/v1/solicitudes/10/aceptar",
  "timestamp": "2026-10-20T13:12:44.512"
}
```

6.2 Códigos HTTP utilizados

Código	Nombre	Cuándo ocurre	Campo error	Ejemplo de mensaje
400	Bad Request	`MethodArgumentNotValidException` (falla de `@Valid`)	Datos de entrada inválidos	Lista de `campo: mensaje`, por ejemplo *"contrasena: La contraseña debe tener al menos 6 caracteres, correo: El formato del correo es inválido"*
400	Bad Request	`HttpMessageNotReadableException`, `MethodArgumentTypeMismatchException`	Formato de datos inválido	"El cuerpo o los parámetros de la petición tienen un formato o valor no válido"
401	Unauthorized	`AuthenticationException` lanzada en un controlador (login)	Error de autenticación	"Bad credentials" o "User is disabled"
403	Forbidden	`AccessDeniedException` de `@PreAuthorize`	Acceso denegado	"No tiene los permisos requeridos para acceder a este recurso."
403	Forbidden	Petición sin token o con token inválido (lo resuelve Spring Security antes del controlador)	—	Respuesta sin cuerpo (ver L2)
404	Not Found	`UsuarioNoEncontradoException`, `RecursoNoEncontradoException`	Recurso no encontrado	"No se encontró ningún usuario con el ID: 42", "Solicitud no existe", "Perfil de trabajador no encontrado con ID: 7"
409	Conflict	`CorreoYaRegistradoException`	Conflicto de unicidad	"El correo contratante@oficioya.com ya se encuentra registrado."
409	Conflict	`ConflictoException`	Conflicto de negocio	"El usuario 2 ya tiene un perfil de trabajador"
422	Unprocessable Entity	`EstadoInvalidoException`	Transición de estado inválida	"No se puede ACEPTAR la solicitud desde el estado CREADA", "Solo se pueden calificar solicitudes finalizadas", "La cuenta ya se encuentra dada de baja"
422	Unprocessable Entity	`ReglaDeNegocioException`	Regla de negocio incumplida	"La tarifa mínima no puede ser mayor que la tarifa máxima", "Hay franjas de disponibilidad solapadas el día MONDAY", "Define tu zona de cobertura antes de activar 'Disponible ahora'"
500	Internal Server Error	Cualquier otra `Exception`	Error interno del servidor	"Ocurrió un error inesperado en el sistema." (sin stack trace; el detalle queda en el log)

6.3 Catálogo de mensajes de negocio

Estos son los textos exactos que devuelve la API en el campo `mensaje`, tomados de las excepciones del código.

Módulo	Mensajes
Usuarios	El correo ya se encuentra verificado en el sistema. · El usuario consultado no es un trabajador. · El usuario consultado no es un contratante. · La cuenta ya se encuentra dada de baja. · No se puede actualizar la foto de una cuenta inactiva. · La cuenta se encuentra inactiva, contacte a soporte.
Archivos	El archivo está vacío. · El archivo debe ser una imagen (JPG, JPEG, PNG).
Perfil	Solo un usuario con rol TRABAJADOR puede tener perfil de trabajador. · La cuenta del usuario está inactiva. · La cuenta debe tener el correo o el teléfono verificado para crear el perfil. · La hora de inicio debe ser anterior a la hora de fin. · Hay franjas de disponibilidad solapadas el día X. · El perfil ya está marcado como disponible ahora. · El perfil no está marcado como disponible ahora. · No se puede asignar un oficio inactivo como principal. · Uno o más oficios secundarios proporcionados no existen en el catálogo.

Módulo	Mensajes
Búsqueda	La tarifa mínima no puede ser mayor que la tarifa máxima. · Para filtrar por hora debes indicar también el día. · Para indicar horaHasta debes indicar también horaDesde. · La hora de inicio debe ser anterior a la hora de fin.
Solicitudes	Solo un contratante puede crear solicitudes. · La cuenta del contratante se encuentra inactiva. · No se puede ENVIAR / ACEPTAR / RECHAZAR / INICIAR / COMPLETAR / CANCELAR la solicitud desde el estado X.
Reputación	Solo se pueden calificar solicitudes finalizadas. · Solo el contratante de la solicitud puede calificar al trabajador. · Solo el trabajador de la solicitud puede calificar al contratante. · La reseña ya había sido reportada.
Referidos	El correo referido ya pertenece a una cuenta existente. · Este correo ya ha sido invitado previamente.

Criterio 409 frente a 422

Usamos **409** cuando el dato choca con otro que ya existe (correo repetido, perfil duplicado) y **422** cuando el dato está bien formado pero rompe una regla del negocio (transición de estado ilegal, tarifa mínima mayor que la máxima, franjas solapadas).

7. Diagramas de secuencia

Uno por funcionalidad principal. Todos muestran el recorrido Controlador → Servicio → Validador → Repositorio → base de datos y, en los bloques *alt*, la ruta de error con el código que llega al cliente. Cada petición protegida entra primero por `JwtAuthenticationFilter`: si el token falta, es inválido o el rol no alcanza, la petición se corta ahí con 403 (sección 10, L2). El detalle del filtro está en 7.2. El validador aparece donde el código tiene uno (`UsuarioValidator`, `PerfilTrabajadorValidator`, `SolicitudValidator` y `BusquedaValidator`); en transiciones, reseñas, moderación y portafolio las reglas viven en el servicio o en la clase de estado, y así se dibujaron.

7.1 Inicio de sesión y emisión del token

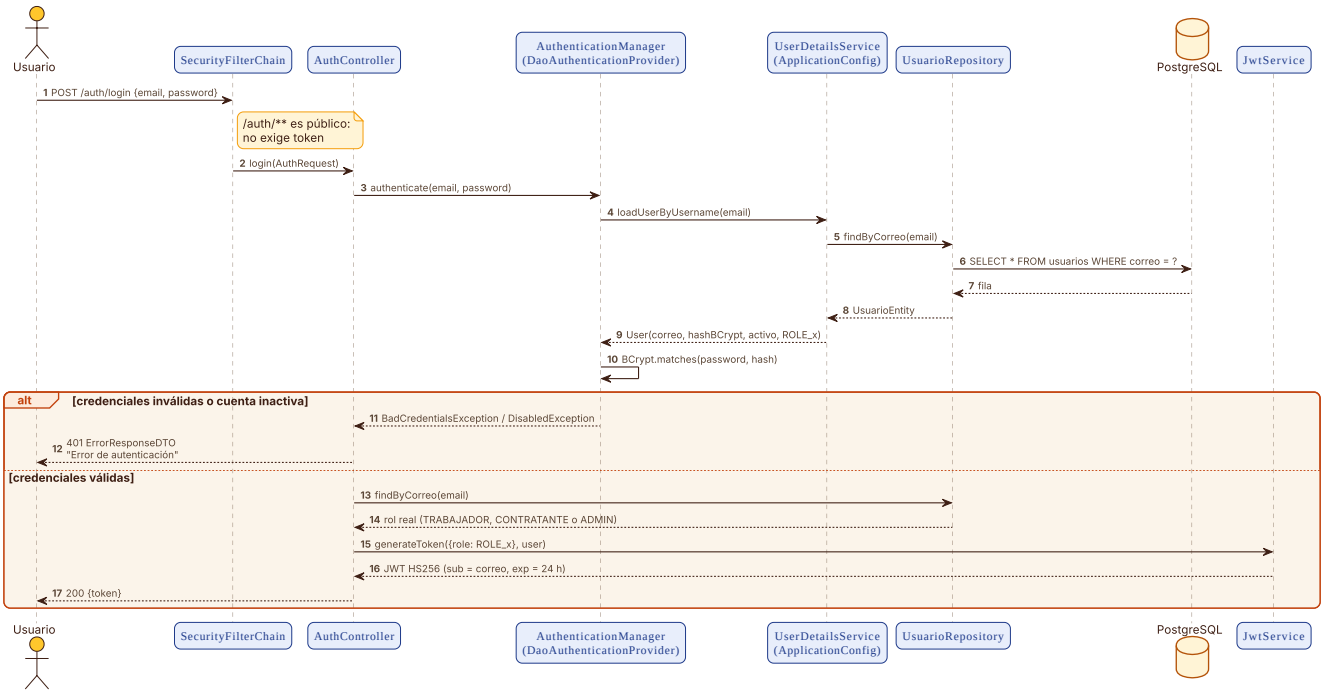


Figura 13. Inicio de sesión.

La ruta es pública. `AuthenticationManager` delega en `DaoAuthenticationProvider`, que carga el usuario por correo y compara la contraseña con BCrypt. Si la cuenta está pausada, `UserDetails.enabled` es falso y el login falla igual que con una contraseña errada (401). Si todo está bien, el token se firma con el rol real leído de la base de datos.

7.2 Petición protegida: filtro JWT y autorización por rol

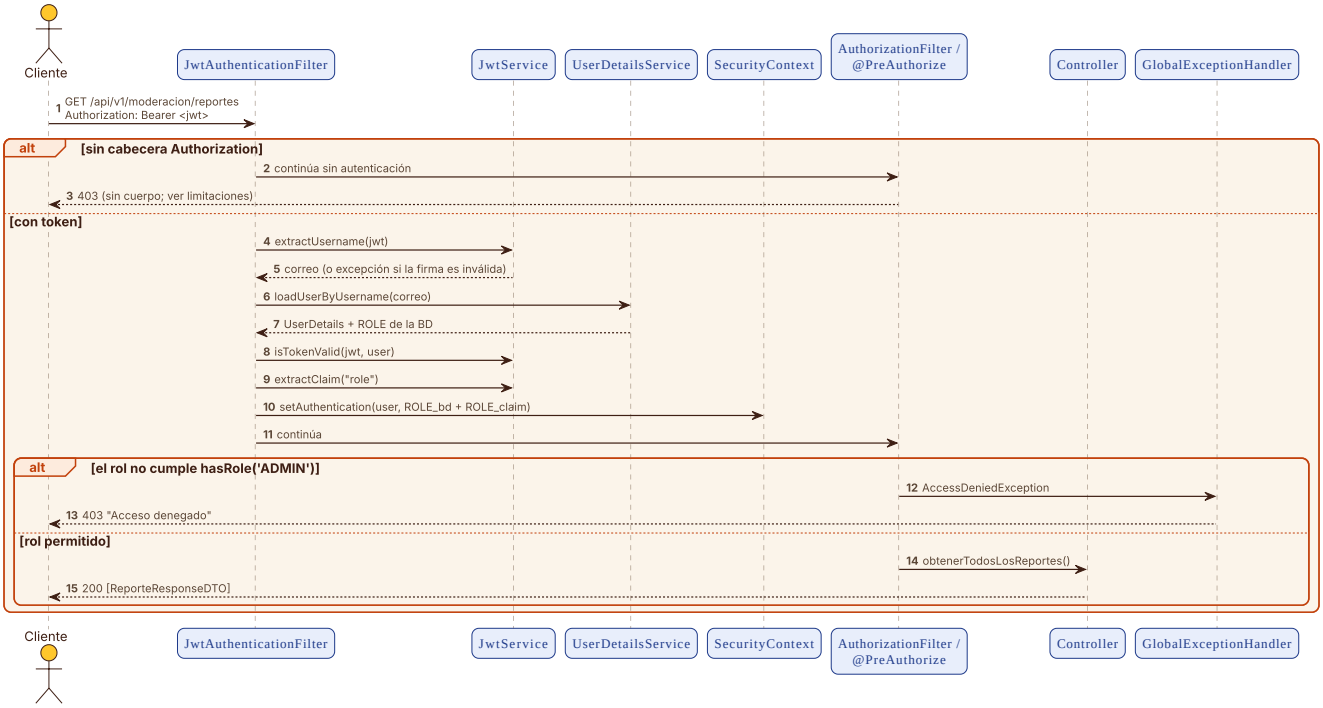


Figura 14. Filtro JWT y @PreAuthorize.

El filtro extrae el correo del token, carga el usuario, valida firma y vencimiento y deja la autenticación en el `SecurityContext`. Después, `@PreAuthorize` compara el rol. Esta secuencia es la que se muestra en vivo en la feria: sin token, con el rol equivocado (403) y con el rol correcto (200).

7.3 Registro de usuario

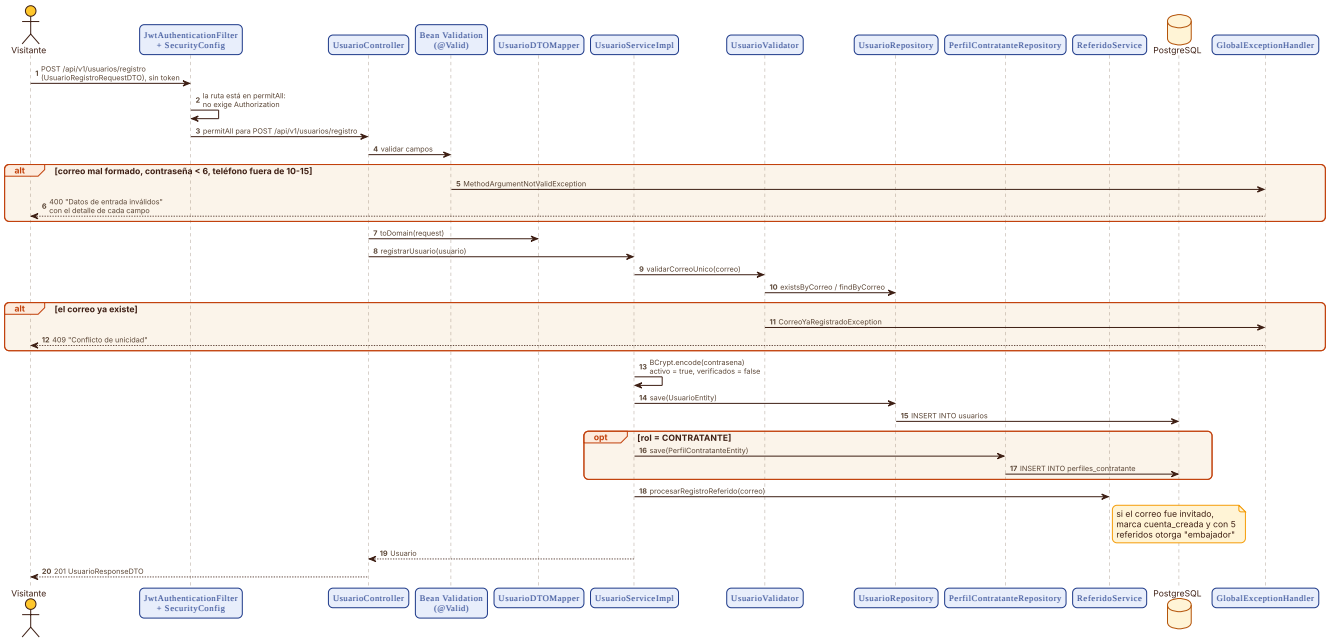


Figura 15. Registro de usuario.

La validación de forma ocurre antes de entrar al servicio (400); la validación de unicidad la hace el validador (409). El registro crea el perfil de contratante cuando corresponde y avisa al módulo de referidos, todo dentro de una sola transacción.

7.4 Creación del perfil profesional

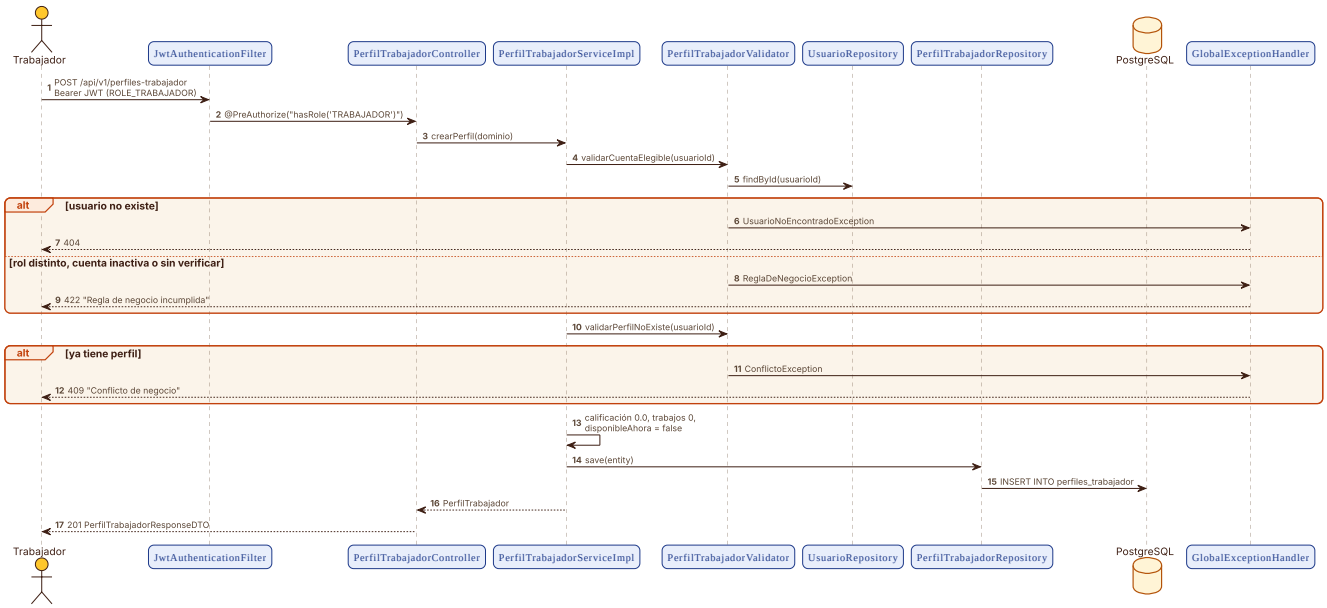


Figura 16. Creación del perfil de trabajador.

El validador revisa tres condiciones de elegibilidad (rol, cuenta activa y correo o teléfono verificado) y que no exista ya un perfil. Cada condición tiene su código: 404, 422 o 409.

7.5 Búsqueda de trabajadores

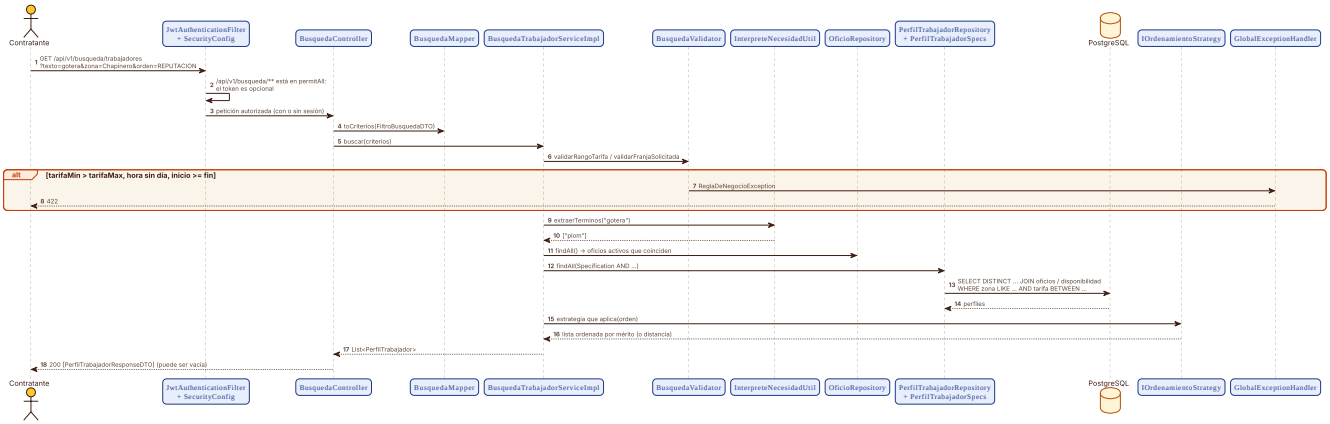


Figura 17. Búsqueda con texto libre, filtros y orden.

El texto libre se interpreta antes de consultar, los filtros se combinan en una sola consulta con `DISTINCT` para no duplicar perfiles por los JOIN, y el orden lo pone la estrategia elegida. Una lista vacía es una respuesta válida (200) que la SPA convierte en un estado vacío con Many.

7.6 Creación y envío de una solicitud

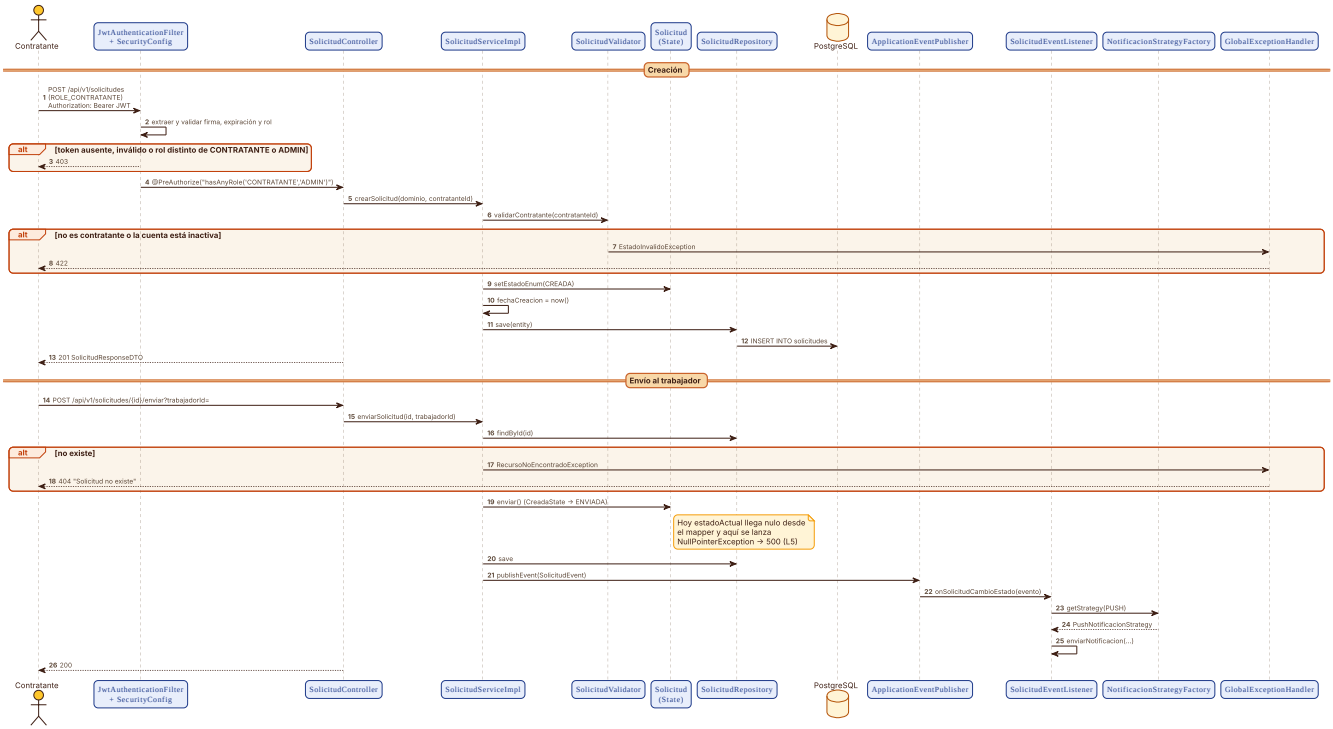


Figura 18. Crear y enviar una solicitud, con el evento de notificación.

La creación deja la solicitud en CREADA. El envío la mueve a ENVIADA mediante su objeto de estado y publica el evento; el listener elige el canal con la fábrica de estrategias. Hoy la creación funciona (201), pero el envío responde 500 por L5, y L6 explica por qué el trabajador no queda asignado.

7.7 Transición de estado (aceptar)

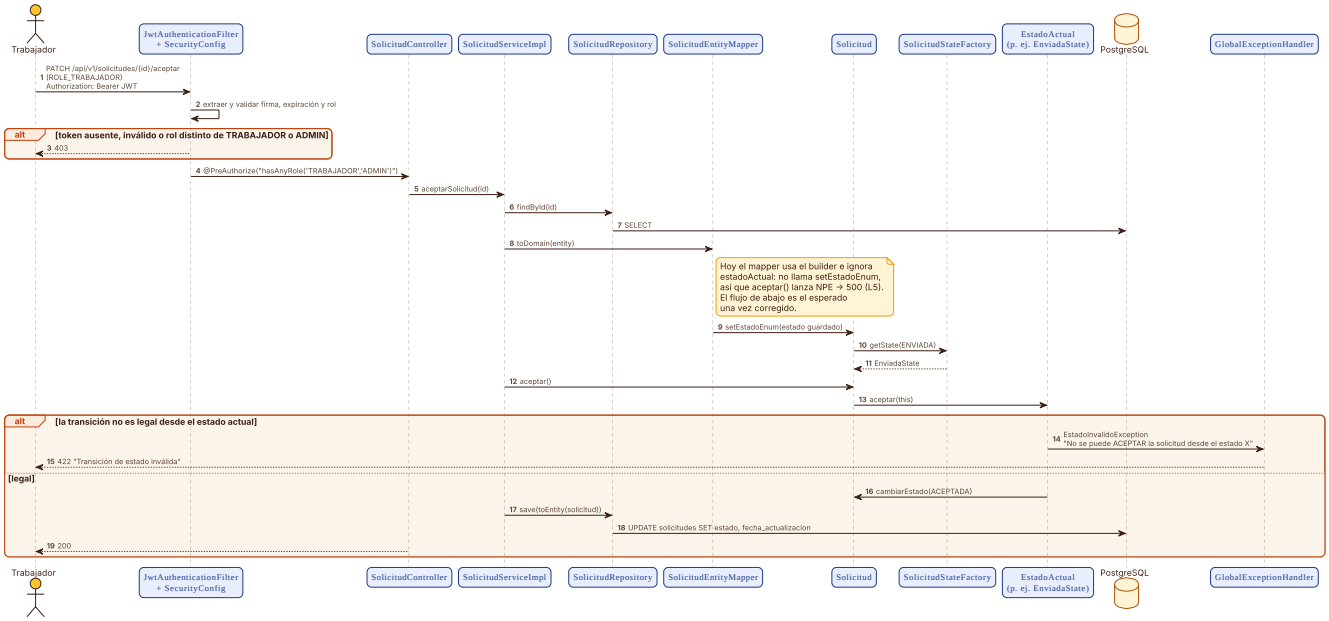


Figura 19. Aceptar una solicitud con el patrón State.

Iniciar, completar, rechazar y cancelar siguen exactamente esta misma secuencia; solo cambia el método invocado sobre el estado. Si la transición no es legal desde el estado actual, el propio objeto de estado lanza la excepción y la API responde 422.

7.8 Calificación del trabajador

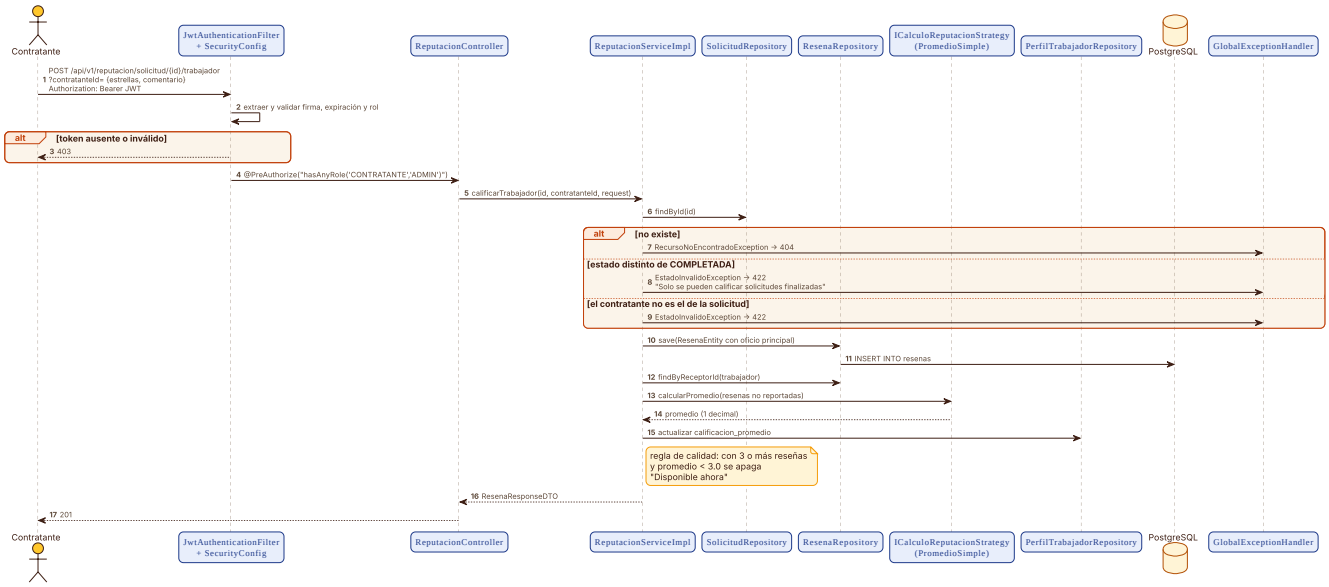


Figura 20. Reseña del contratante al trabajador y recálculo de la reputación.

Solo se puede calificar una solicitud COMPLETADA y solo puede hacerlo su contratante. Al guardar la reseña se recalcula el promedio con la estrategia activa y se aplica la regla de calidad sobre "Disponible ahora". La calificación del contratante por parte del trabajador es simétrica.

7.9 Reporte y moderación

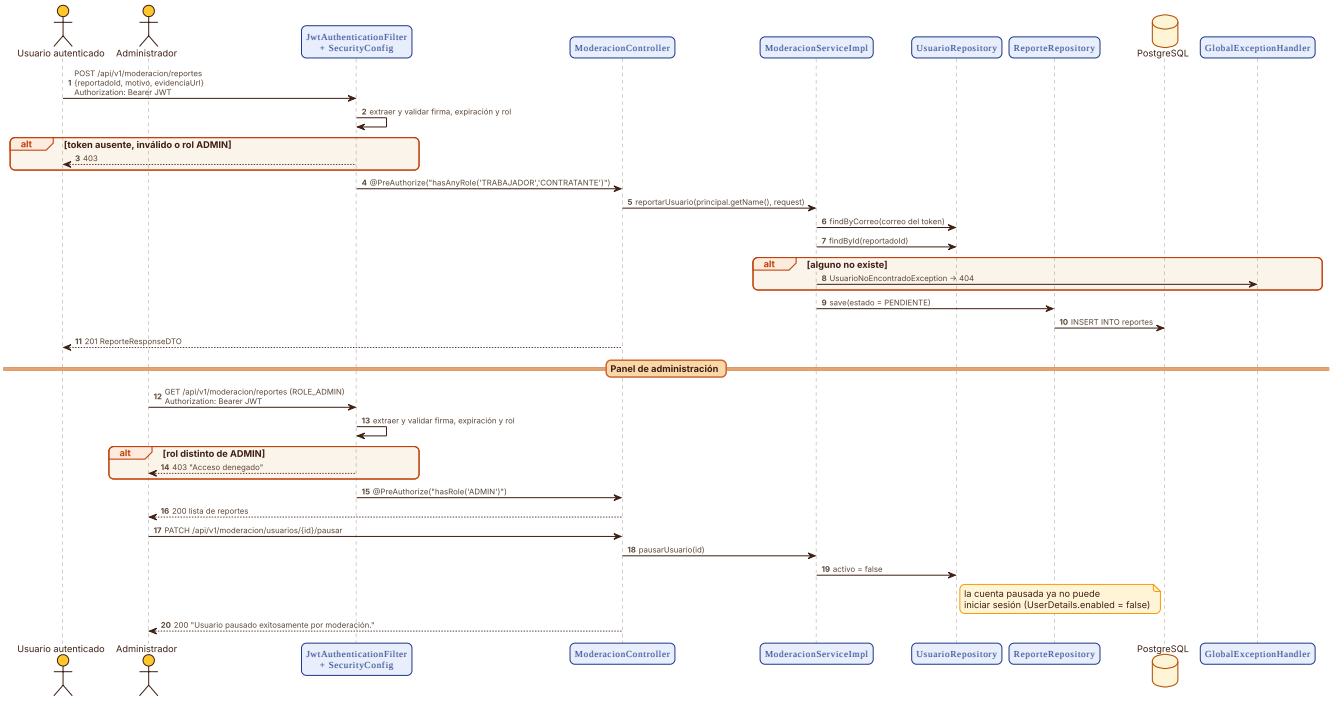


Figura 21. Reportar un usuario y moderar desde el panel de administración.

El reportador se toma del token, no del cuerpo, así nadie puede reportar a nombre de otro. El administrador lista los reportes y puede pausar la cuenta; desde ese momento el usuario no puede iniciar sesión.

7.10 Actualización del portafolio (PostgreSQL + MongoDB)

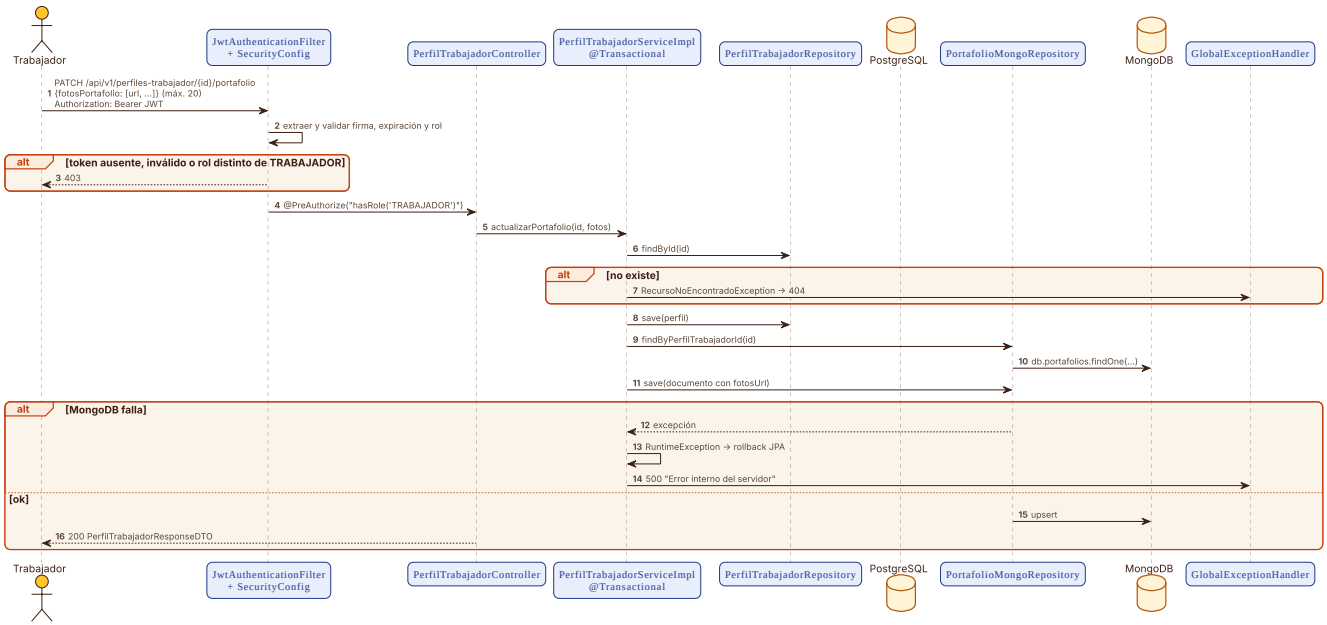


Figura 22. Portafolio con persistencia híbrida y compensación.

Es el único flujo que escribe en los dos motores. Si MongoDB falla, la excepción revierte la transacción relacional para no dejar el perfil a medias.

8. Despliegue y seguridad

8.1 Diagrama de despliegue

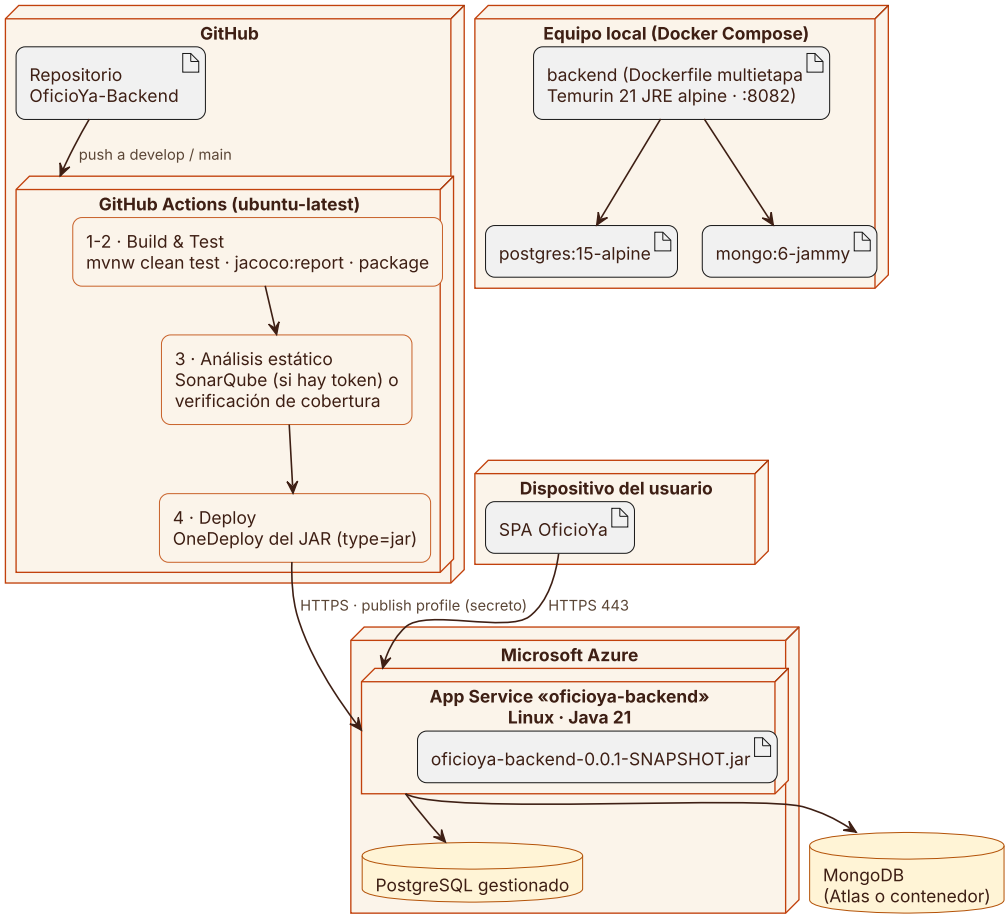


Figura 23. Despliegue en Azure, pipeline de GitHub Actions y entorno local con Docker Compose.

Cada push a `develop` o `main` dispara el pipeline `azure-deploy.yml` con cuatro etapas encadenadas: compilación y pruebas con JDK 21 Temurin y reporte de JaCoCo; análisis estático (SonarQube cuando existen los secretos `SONAR_TOKEN` y `SONAR_HOST_URL`, o una verificación de cobertura desde el CSV de JaCoCo cuando no); y despliegue del JAR al App Service `oficioya-backend` por OneDeploy, autenticado con el perfil de publicación guardado como secreto. Si el despliegue no devuelve 200 o 202, el pipeline falla.

Para desarrollo local, `docker compose up --build -d` levanta PostgreSQL 15, MongoDB 6 y el backend construido con un Dockerfile de dos etapas (Maven para compilar y Temurin 21 JRE Alpine para ejecutar, zona horaria America/Bogota, perfil `docker`, puerto 8082).

8.2 URLs

Recurso	URL
API (Azure)	https://oficioya-backend.azurewebsites.net
Swagger UI	https://oficioya-backend.azurewebsites.net/swagger-ui/index.html
Contrato OpenAPI	https://oficioya-backend.azurewebsites.net/v3/api-docs
Salud	https://oficioya-backend.azurewebsites.net/api/v1/health
Local (Maven o Docker)	http://localhost:8082/swagger-ui/index.html
Repositorio	https://github.com/OficioYa1/OficioYa-Backend

8.3 Configuración por variables de entorno

Variable	Valor por defecto	Uso
`SERVER_PORT`	8082	Puerto HTTP
`SPRING_DATASOURCE_URL` / `_USERNAME` / `_PASSWORD`	jdbc:postgresql://localhost:5432/oficioya_db	Conexión a PostgreSQL
`SPRING_DATA_MONGODB_URI`	mongodb://localhost:27017/oficioya_mongo_db	Conexión a MongoDB
`APPLICATION_SECURITY_JWT_SECRET_KEY`	clave de desarrollo (ver L12)	Clave HMAC del JWT, en Base64

Variable	Valor por defecto	Uso
`APPLICATION_SECURITY_JWT_EXPIRATION`	86400000 (24 h)	Vida del token en milisegundos
`SPRING_JPA_HIBERNATE_DDL_AUTO`	update	Gestión del esquema
`spring.servlet.multipart.max-file-size`	5 MB	Tamaño máximo de archivos

8.4 Modelo de seguridad

- **Sin estado:** `SessionCreationPolicy.STATELESS` y CSRF deshabilitado, porque no hay cookies de sesión.
- **Contraseñas:** BCrypt al registrarse y en los usuarios semilla.
- **Token:** HS256, `sub` = correo, claim `role`, 24 horas de vida.
- **Cuenta pausada o dada de baja:** `enabled = false`, no puede iniciar sesión.
- **Autorización:** `@EnableMethodSecurity` y `@PreAuthorize` en cada método de controlador protegido.
- **Usuarios de demostración** (los crea el `DataSeeder` si la tabla está vacía): `admin@oficioya.com`, `trabajador@oficioya.com` y `contratante@oficioya.com`, todos con la contraseña de demostración del seeder.

8.5 Matriz de roles y permisos

Recurso	Anónimo	Contratante	Trabajador	Admin
Autenticación (`/auth/**`)	✓	✓	✓	✓
Salud y Swagger	✓	✓	✓	✓
Registro, recuperar cuenta, catálogo y búsqueda	✓	✓	✓	✓
Verificar correo, consultar perfiles	×	✓	✓	✓
Editar perfil de trabajador	×	×	✓	✓
Editar perfil de contratante	×	✓	×	✓
Eliminar cuenta, subir foto	×	✓	✓	×
Crear y configurar perfil profesional	×	×	✓	×
Ver perfil profesional por id	×	✓	✓	✓
Verificación de identidad	×	×	✓	✓
Crear, enviar y cancelar solicitud	×	✓	×	✓
Consultar una solicitud por id	×	✓	✓	✓
Aceptar, rechazar, iniciar y completar	×	×	✓	✓
Mensajería	×	✓	✓	✓
Historial y estadísticas del trabajador	×	×	✓	×
Historial y estadísticas del contratante	×	× hoy (L4)	×	×
Calificar al trabajador	×	✓	×	✓
Calificar al contratante	×	×	✓	✓
Reportar reseña, consultar reputación	×	✓	✓	✓
Reportar usuario, invitar referidos	×	✓	✓	×
Consultar un reporte	×	✓	✓	✓
Listar reportes y pausar cuentas	×	×	×	✓

9. Calidad y evidencias

9.1 Pruebas automatizadas

El 20 de octubre de 2026 corrimos `./mvnw test jacoco:report` sobre el commit `890b8b8` de `develop`. Resultado: **BUILD SUCCESS**.

338	0	95,3 %	94,9 %
pruebas ejecutadas	fallos y errores	cobertura de instrucciones	cobertura de líneas

Métrica JaCoCo	Cubiertas / total	Porcentaje
Instrucciones	8.890 / 9.332	95,26 %
Ramas	429 / 554	77,44 %
Líneas	1.867 / 1.967	94,92 %
Métodos	445 / 470	94,68 %
Clases analizadas	105	—

El `pom.xml` exige un mínimo de 85 % de instrucciones. Las pruebas siguen el patrón Arrange-Act-Assert y cubren controladores, servicios, validadores, mappers, estados, estrategias, la cadena de búsqueda, los decoradores, el filtro JWT y una prueba de integración de seguridad con MockMvc (sin token, rol incorrecto y rol correcto).

9.2 Pruebas de funcionamiento contra la aplicación levantada

Para este documento levantamos el JAR del mismo commit en local, con H2 en lugar de PostgreSQL, y recorrimos los casos de la tabla. No fue posible llamar la URL de Azure desde el entorno donde se preparó el documento, así que la evidencia contra PROD debe tomarse con la colección de Postman del equipo. Las filas en rojo son las limitaciones de la sección 10.

ID	Funcionalidad	Escenario	Esperado	Obtenido
EV-01	Salud	GET /api/v1/health	200	✅ 200 {"status":"UP"}
EV-02	OFY-50 Login	POST /auth/login con credenciales válidas	200 + token	✅ 200 con JWT
EV-03	OFY-50 Login	POST /auth/login con contraseña incorrecta	401	✅ 401 "Error de autenticación"
EV-04	Seguridad	GET /api/v1/moderacion/reportes sin token	401	⚠ 403 sin cuerpo (L2)
EV-05	Seguridad	GET /api/v1/moderacion/reportes con rol CONTRATANTE	403	✅ 403 "Acceso denegado"
EV-06	OFY-77 Catálogo	GET /api/v1/oficios sin token	200	✅ 200 con los 18 oficios
EV-07	Registro	POST /api/v1/usuarios/registro con datos inválidos	400	✅ 400 con el detalle por campo
EV-08	Registro	POST /api/v1/usuarios/registro con correo repetido	409	✅ 409 "Conflicto de unicidad"
EV-09	Solicitudes	PATCH /api/v1/solicitudes/999/cancelar	404	✅ 404 "Solicitud no existe"
EV-10	Registro	POST /api/v1/usuarios/registro sin token	201	✅ 201 con el usuario creado
EV-11	Búsqueda	GET /api/v1/busqueda/trabajadores sin token	200	✅ 200
EV-12	Historial	GET /historial-estadisticas/cliente/3/historial como CONTRATANTE	200	❌ 403 (L4)
EV-13	OFY-55	POST /auth/switch-role con newRole = ADMIN desde un contratante	403	❌ 200 y el token nuevo abre endpoints de ADMIN (L3)
EV-14	Solicitudes	POST /api/v1/solicitudes con datos válidos	201	✅ 201 en estado CREADA y con fechaCreacion
EV-15	Solicitudes	GET /api/v1/solicitudes/1 como CONTRATANTE	200	✅ 200 con la solicitud
EV-16	Solicitudes	GET /api/v1/solicitudes/999	404	✅ 404 "Solicitud no existe"
EV-17	Solicitudes	POST /api/v1/solicitudes/1/enviar sobre una solicitud CREADA	200	❌ 500 por `estadoActual` nulo (L5)

10. Limitaciones conocidas

Declararlas es parte del trabajo. Las clasificamos con la misma escala del sprint: P1 bloquea la demo, P2 se ve en la feria y P3 es calidad interna. Cada una dice qué falta, por qué importa y cómo la resolveríamos.

L2 P2 Sin token responde 403 en vez de 401

Qué pasa: No hay un `AuthenticationEntryPoint` configurado, así que Spring Security usa el de por defecto (403 sin cuerpo). Un token mal formado o vencido también termina en 403, porque la excepción de JJWT ocurre en el filtro, antes del `GlobalExceptionHandler`.

Impacto: El front no puede distinguir "inicia sesión de nuevo" de "no tienes permiso". La prueba de seguridad del sprint espera 401.

Cómo lo resolveríamos: Configurar `exceptionHandling().authenticationEntryPoint(...)` que escriba un `ErrorResponseDTO` con 401, y capturar `JwtException` dentro del filtro.

L3 P1 El cambio de rol permite pedir ADMIN

Qué pasa: `/auth/switch-role` firma un token con el rol que llegue en el cuerpo y el filtro suma ese rol a las autoridades. Se comprobó localmente: un contratante pide `ADMIN` y con el token nuevo lista los reportes de moderación (200).

Impacto: Escalamiento de privilegios: cualquier usuario puede volverse administrador.

Cómo lo resolveríamos: Aceptar solo TRABAJADOR o CONTRATANTE, validar que el usuario tenga perfil para ese rol y nunca emitir ADMIN desde este endpoint.

L4 P2 Rol CLIENTE inexistente en historial

Qué pasa: Los tres endpoints de historial, estadísticas y tablero del contratante usan `hasRole('CLIENTE')`, pero el enum `RolUsuario` solo tiene CONTRATANTE, TRABAJADOR y ADMIN.

Impacto: El contratante siempre recibe 403 en su historial y su tablero.

Cómo lo resolveríamos: Reemplazar por `hasAnyRole('CONTRATANTE','ADMIN')`.

L5 P1 Las transiciones de una solicitud guardada responden 500

Qué pasa: `SolicitudEntityMapper.toDomain` ignora `estadoActual` y, como `Solicitud` tiene `@Builder`, MapStruct arma el objeto con el builder y no pasa por `setEstadoEnum`, que es el que crea el objeto de estado. Al leer la solicitud de la base, `estadoActual` queda en `null` y `enviar()`, `aceptar()` y las demás transiciones lanzan `NullPointerException`. Se reprodujo localmente: `POST /solicitudes/1/enviar` y `PATCH /solicitudes/1/aceptar` responden 500. Las pruebas unitarias no lo detectan porque simulan el repositorio.

Impacto: Una solicitud se puede crear y consultar, pero no avanza de CREADA. El ciclo de vida completo no se puede mostrar por la API.

Cómo lo resolveríamos: Llamar `solicitud.setEstadoEnum(entity.getEstado())` al recuperar el dominio (o un `@AfterMapping` en el mapper) y agregar una prueba de integración que cree, envíe y acepte una solicitud.

L6 P2 Datos que se reciben y no se guardan

Qué pasa: `enviarSolicitud` recibe `trabajadorId` pero no lo asigna a la solicitud, y `cancelarSolicitud` recibe `motivo` pero no lo persiste. `ReputacionPorOficioDTO.totalResenas` cuenta todas las reseñas del trabajador, no solo las del oficio (ver también L10).

Impacto: El trabajador asignado no queda registrado, por lo que las reseñas y el historial del trabajador no encuentran la solicitud.

Cómo lo resolveríamos: Asignar el trabajador al enviar, agregar columna `motivo_cancelacion` y filtrar el conteo por oficio.

L7 P2 Identidad tomada de parámetros y no del token

Qué pasa: Calificar (`contratanteId`, `trabajadorId`), chatear (`remitenteId`) y editar perfiles por `{id}` confían en el valor que envía el cliente; el código lo marca como "simulación de sesión". Moderación y referidos sí usan el `Principal`.

Impacto: Un usuario autenticado podría actuar en nombre de otro.

Cómo lo resolveríamos: Resolver el usuario desde `Principal` (correo del token) y validar la propiedad del recurso en el servicio.

L8 P2 Swagger y respuestas reales no coinciden en algunos casos

Qué pasa: Swagger documenta 400 para la foto inválida y para calificar una solicitud no finalizada, pero la API responde 422. Los parámetros obligatorios de query que faltan (`motivo`, `trabajadorId`) caen en el manejador genérico y responden 500.

Impacto: Un jurado que compare Swagger con la respuesta real verá la diferencia.

Cómo lo resolveríamos: Ajustar las anotaciones `@ApiResponse` y manejar `MissingServletRequestParameterException` como 400.

L9 P3 Integraciones externas simuladas

Qué pasa: `StorageServiceImpl` devuelve URLs `mock.cloudinary.com`, `MockMapboxAdapter` calcula una distancia aproximada a partir de los nombres de zona, y las estrategias de notificación solo registran en el log. La cadena de búsqueda por zona (Chain of Responsibility) y el Builder + Decorator de insignias están implementados y probados, pero el endpoint público de búsqueda usa las `Specification` y no los invoca.

Impacto: La demo funciona, pero las imágenes no se guardan de verdad y el orden por distancia es ilustrativo.

Cómo lo resolveríamos: Conectar los adaptadores reales detrás de las mismas interfaces (`IStorageService`, `IMapAdapter`, `INotificacionStrategy`) sin tocar los servicios.

L10 P3 Detalles de la reputación por oficio

Qué pasa: `oficioNombre` se arma como "Oficio {id}" en lugar de leer el catálogo, y la reseña siempre se asocia al oficio principal del trabajador.

Impacto: La reputación por oficio es aproximada cuando el trabajador tiene varios oficios.

Cómo lo resolveríamos: Leer el nombre desde `OficioRepository` y guardar en la solicitud el oficio contratado.

L11 P3 Un solo ambiente y pipeline simplificado

Qué pasa: QA y PROD apuntan a la misma URL. El pipeline despliega el JAR con OneDeploy; no publica imagen en Docker Hub ni tiene aprobación manual. Si no existe `SONAR_TOKEN`, la etapa de SonarQube solo calcula la cobertura.

Impacto: No hay un ambiente donde probar antes de producción.

Cómo lo resolveríamos: Crear un segundo App Service para QA, usar GitHub Environments con aprobación para PROD y publicar la imagen Docker en cada versión.

L12 P3 Configuración sensible por defecto

Qué pasa: La clave JWT tiene un valor por defecto en `application.properties`; el perfil Docker define `jwt.secret`, que `JwtService` no lee. CORS acepta cualquier origen con credenciales y el perfil por defecto imprime el SQL.

Impacto: Riesgo bajo en el contexto académico, pero no apto para producción real.

Cómo lo resolveríamos: Exigir `APPLICATION_SECURITY_JWT_SECRET_KEY` sin valor por defecto, restringir CORS al dominio del front y apagar `show-sql`.

L13 P2 El código final no está integrado en main

Qué pasa: El estado descrito aquí ya está en `develop` (PR #36, commit `890b8b8`), pero `main` conserva el esqueleto inicial y va 193 commits detrás. Además conviven las carpetas `com/oficioYa` y `com/oficioya` para el mismo paquete.

Impacto: Si el despliegue o la entrega se toman de `main`, no coinciden con este documento.

Cómo lo resolveríamos: Abrir el PR de `develop` hacia `main` y unificar las carpetas en minúscula.

L14 P2 Requerimientos del análisis sin endpoint

Qué pasa: 14 de los 78 requerimientos del análisis no tienen endpoint o lo tienen a medias: expiración a los 30 minutos, envío a varios trabajadores, ampliación de zona, ubicación del dispositivo, ingresos y servicios más solicitados, verificación tras 10 trabajos e insignia de identidad, entre otros. El detalle está en la sección 3.1.

Impacto: El jurado puede preguntar por una historia del análisis que el backend no expone.

Cómo lo resolveríamos: Priorizar OFY-70 y OFY-22 (solo hay que llamar `buscarTrabajadores(zona, …)` desde el endpoint de búsqueda) y OFY-28 (una tarea `@Scheduled` con un estado EXPIRADA).

Orden sugerido

Si hay una ventana para tocar el código, el orden con mayor efecto en la demo es L5, L3 y L6 (cambios pequeños en `SolicitudServiceImpl` y `AuthController`), luego L4 y L2. El resto puede quedar documentado para la siguiente versión.

11. Bibliografía

1. Broadcom (2026). *Spring Boot Reference Documentation*, versión 3.4. https://docs.spring.io/spring-boot/
2. Broadcom (2026). *Spring Security Reference*, versión 6.4. https://docs.spring.io/spring-security/reference/
3. Broadcom (2026). *Spring Data JPA Reference* (Specifications). https://docs.spring.io/spring-data/jpa/reference/
4. Broadcom (2026). *Spring Data MongoDB Reference*. https://docs.spring.io/spring-data/mongodb/reference/
5. Jones, M., Bradley, J. y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF. https://www.rfc-editor.org/rfc/rfc7519
6. Fielding, R., Nottingham, M. y Reschke, J. (2022). *RFC 9110: HTTP Semantics*. IETF. https://www.rfc-editor.org/rfc/rfc9110
7. JJWT (2023). *Java JWT: JSON Web Token for Java and Android*. https://github.com/jwtk/jjwt
8. springdoc.org (2025). *springdoc-openapi*. https://springdoc.org/
9. MapStruct (2023). *MapStruct Reference Guide* 1.5. https://mapstruct.org/documentation/
10. The PostgreSQL Global Development Group (2025). *PostgreSQL 15 Documentation*. https://www.postgresql.org/docs/15/
11. MongoDB, Inc. (2025). *MongoDB Manual*, versión 6.0. https://www.mongodb.com/docs/manual/
12. Microsoft (2026). *Azure App Service documentation*. https://learn.microsoft.com/azure/app-service/
13. GitHub (2026). *GitHub Actions documentation*. https://docs.github.com/actions
14. EclEmma (2023). *JaCoCo Java Code Coverage Library*. https://www.jacoco.org/jacoco/
15. Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
16. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
17. Brown, S. (2018). *The C4 model for visualising software architecture*. https://c4model.com/
18. PlantUML (2024). *PlantUML Language Reference Guide*. https://plantuml.com/
19. Equipo OficioYa (2026). *Manual de identidad de OficioYa* y *README del frontend*. Repositorio OficioYa-Frontend.



OficioYa · El talento de tu zona a un click de distancia.
Documento de arquitectura del backend · versión 1.0 · 20 de octubre de 2026.